



Deployments, Jobs, and Scaling



Learn how to ...

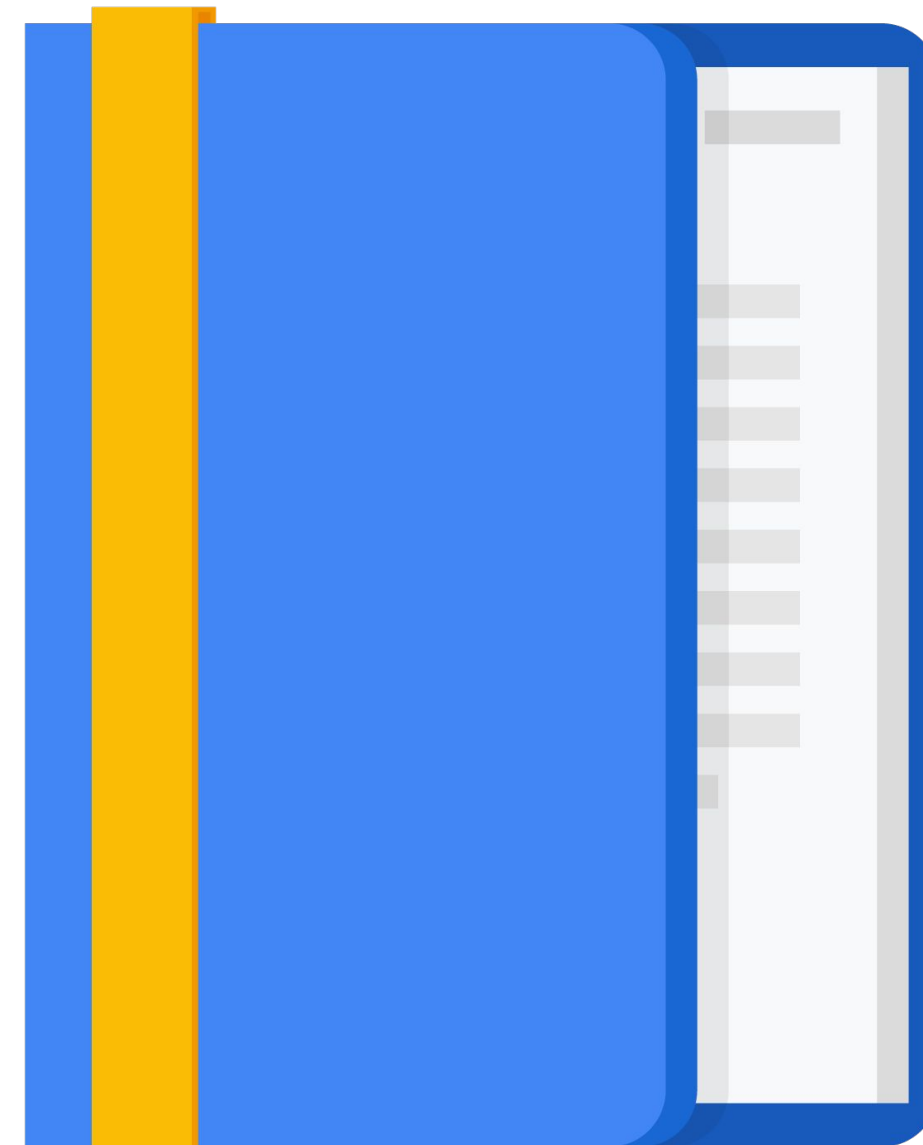
Create and use Deployments.

Create and run Jobs and CronJobs.

Scale clusters manually and automatically.

Configure Node and Pod affinity.

Get software into your cluster.



Agenda

Deployments

Lab: Creating Google Kubernetes Engine Deployments

Jobs and CronJobs

Lab: Deploying Jobs on Google Kubernetes Engine

Cluster Scaling

Controlling Pod Placement

Getting Software into Your Cluster

Lab: Configuring Pod Autoscaling and Node Pools

Summary

Deployments declare the state of Pods



Roll out updates to
the Pods



Roll back Pods to
previous revision

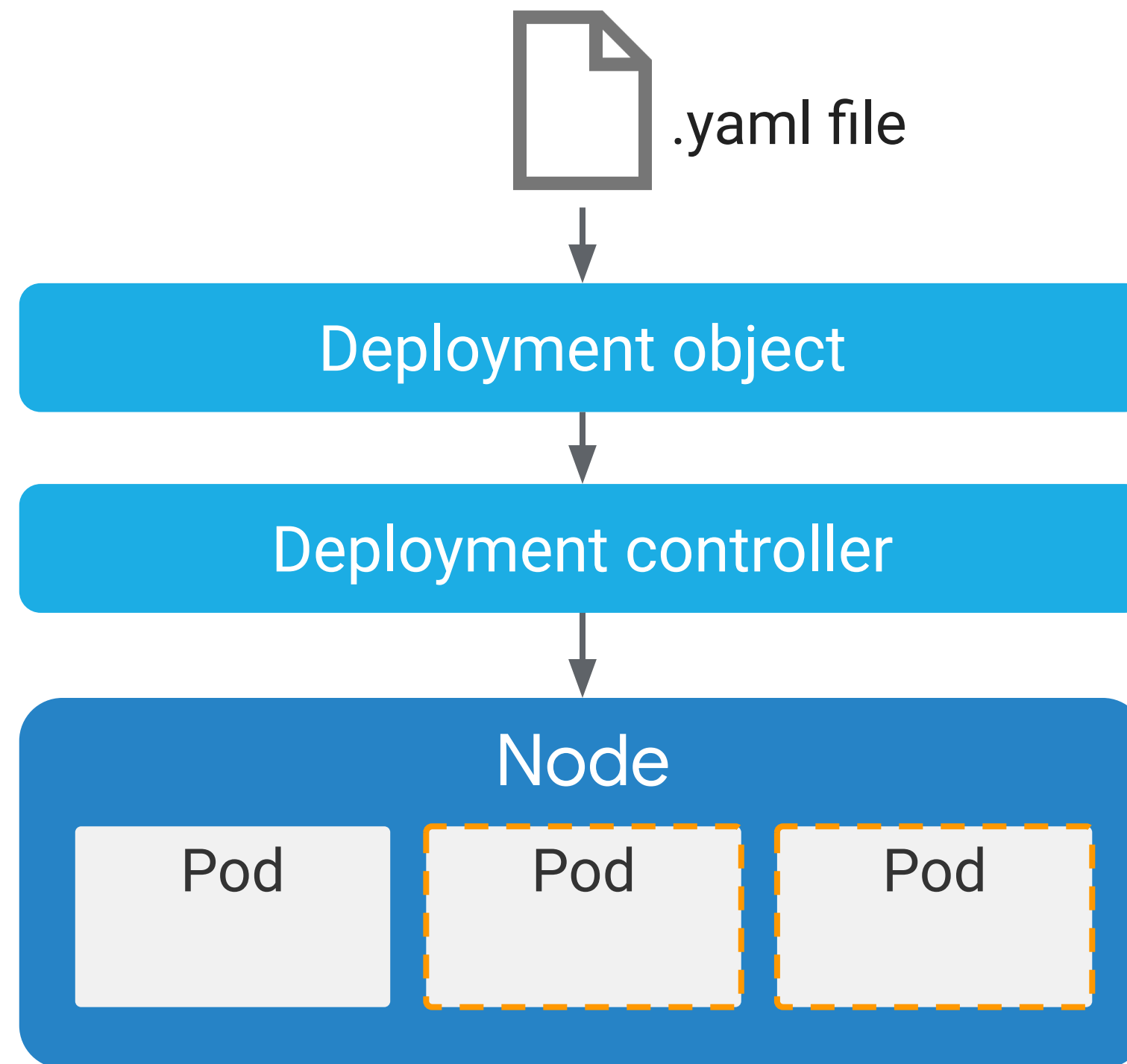


Scale or autoscale
Pods



Well-suited for
stateless applications

Deployment is a two-part process



Deployment object file in YAML format

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: gcr.io/demo/my-app:1.0
        ports:
        - containerPort: 8080
```

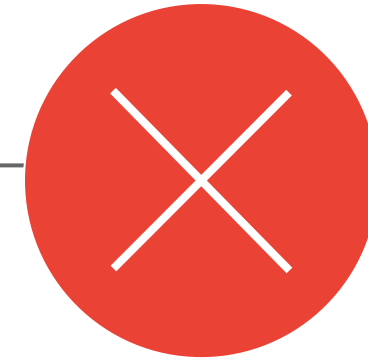
Deployment has three different lifecycle states



Progressing
State



Complete
State



Failed
State

There are three ways to create a Deployment

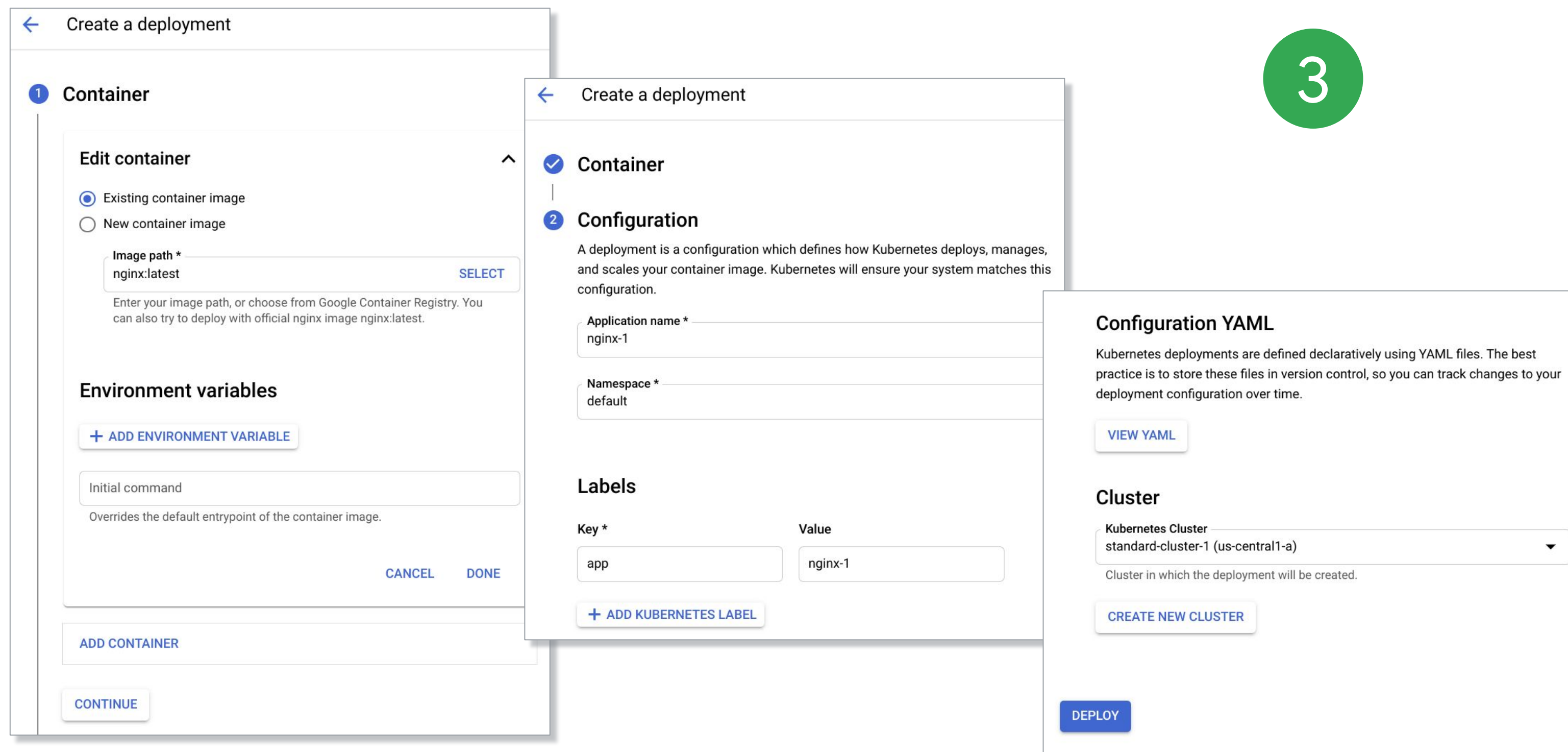
1

```
$ kubectl apply -f [DEPLOYMENT_FILE]
```

2

```
$ kubectl create deployment \  
[DEPLOYMENT_NAME] \  
  --image [IMAGE]:[TAG] \  
  --replicas 3 \  
  --labels [KEY]=[VALUE] \  
  --port 8080 \  
  --generator deployment/apps.v1 \  
  --save-config
```


There are three ways to create a Deployment



← Create a deployment

1 Container

Edit container

☒ Existing container image
☐ New container image

Image path *
nginx:latest [SELECT](#)

Enter your image path, or choose from Google Container Registry. You can also try to deploy with official nginx image nginx:latest.

Environment variables

[+ ADD ENVIRONMENT VARIABLE](#)

Initial command
Overrides the default entrypoint of the container image.

[CANCEL](#) [DONE](#)

[ADD CONTAINER](#)

[CONTINUE](#)

← Create a deployment

☒ **Container**

2 Configuration

A deployment is a configuration which defines how Kubernetes deploys, manages, and scales your container image. Kubernetes will ensure your system matches this configuration.

Application name *
nginx-1

Namespace *
default

Labels

Key *	Value
app	nginx-1

[+ ADD KUBERNETES LABEL](#)

Configuration YAML

Kubernetes deployments are defined declaratively using YAML files. The best practice is to store these files in version control, so you can track changes to your deployment configuration over time.

[VIEW YAML](#)

Cluster

Kubernetes Cluster
standard-cluster-1 (us-central1-a) ▼

Cluster in which the deployment will be created.

[CREATE NEW CLUSTER](#)

[DEPLOY](#)

3

Use `kubectl` to inspect your Deployment, or output the Deployment config in a YAML format

```
$ kubectl get deployment [DEPLOYMENT_NAME]
```

```
master $ kubectl get deployment nginx-deployment
NAME                DESIRED    CURRENT    UP-TO-DATE    AVAILABLE    AGE
nginx-deployment    3          3          3             3            3m
```

```
$ kubectl get deployment [DEPLOYMENT_NAME] -o yaml > this.yaml
```

Use the 'describe' command to get detailed info

```
$ kubectl describe deployment [DEPLOYMENT_NAME]
```

```
master $ kubectl describe deployment nginx-deployment
Name:                nginx-deployment
Namespace:           default
CreationTimestamp:    Fri, 12 Oct 2018 15:23:46 +0000
Labels:              app=nginx
Annotations:         deployment.kubernetes.io/revision=1
Selector:            app=nginx
Replicas:            3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:        RollingUpdate
MinReadySeconds:     0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx
  Containers:
    nginx:
      Image:      nginx:1.15.4
      Port:       80/TCP
      Host Port:  0/TCP
```

Or use the Cloud Console

✓ nginx-deployment

1

To let others access your deployment, expose it to create a service

OVERVIEW

DETAILS

REVISION HISTORY

EVENTS

LOGS

YAML

CPU ?

0.002

0.001

0

Memory ?

Cluster

standard-cluster-1

Namespace

default

Labels

app: nginx

Logs ?

Container logs, Audit logs

Replicas

3 updated, 3 ready, 3 available, 0 unavailable

Pod specification

Revision 1, containers: [nginx](#)

Active revisions

Revision ↓	Name	Status	Summary	Created on	Pods running/total
1	nginx-deployment-5d59d67564	✓ OK	nginx: nginx:1.7.9	Oct 13, 2021, 1:16:02 PM	3/3

Managed pods

Revision	Name	Status	Restarts	Created on ↑
1	nginx-deployment-5d59d67564-2dsfb	✓ Running	0	Oct 13, 2021, 1:16:02 PM
1	nginx-deployment-5d59d67564-8cknj	✓ Running	0	Oct 13, 2021, 1:16:02 PM
1	nginx-deployment-5d59d67564-d898b	✓ Running	0	Oct 13, 2021, 1:16:02 PM

✓ nginx-deployment

1

To let others access your deployment, expose it to create a service

OVERVIEW

DETAILS

REVISION HISTORY

EVENTS

LOGS

YAML

Cluster

standard-cluster-1

Namespace

default

Created

Oct 13, 2021, 1:16:02 PM

Labels

app: nginx

Annotations

deployment.kubernetes.io/revision: 1

SHOW ALL ANNOTATIONS

Replicas

3 updated, 3 ready, 3 available, 0 unavailable

Label selector

app = nginx

Update strategy ?

Rolling update, Max unavailable: 25%, Max surge: 25%

Min time ready before available

0 s

Progress deadline

600 s

Revision history limit

10

Pod specification

Revision 1

Labels

app: nginx

Termination grace period

30

Restart policy

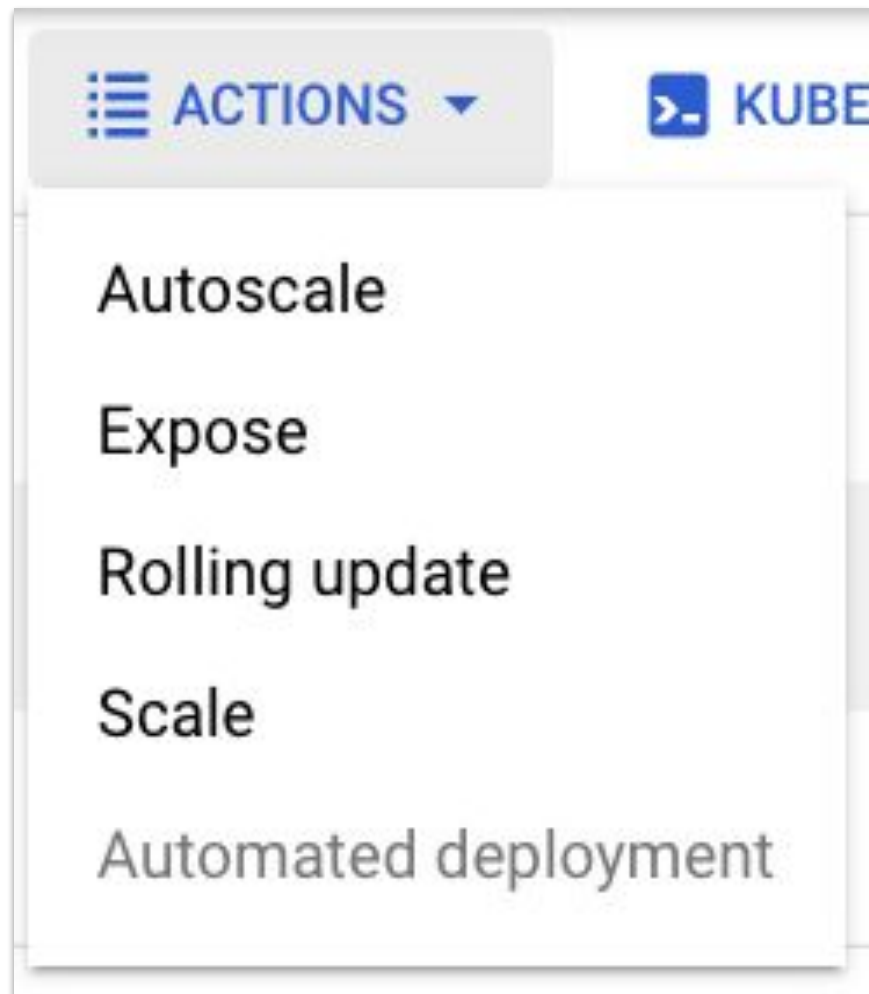
Always

Containers

nginx

You can scale the Deployment manually

```
$ kubectl scale deployment [DEPLOYMENT_NAME] --replicas=5
```



Scale

Scale a workload to a new size.

Replicas *

* Indicates required field

CANCEL **SCALE**

You can also autoscale the Deployment

```
$ kubectl autoscale deployment [DEPLOYMENT_NAME] --min=5 --max=15  
--cpu-percent=75
```

Autoscale

Automatically scale the number of pods.

Minimum number of Pods (Optional)

Maximum number of Pods

Target CPU utilization in percent (Optional)

[CANCEL](#) [DISABLE AUTOSCALER](#) [AUTOSCALE](#)

Thrashing is a problem with any type of autoscaling

Cooldown/delay support:

```
--horizontal-pod-autoscaler-downscale-stabilization
```

You can update a Deployment in different ways

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: my-app
          image: gcr.io/demo/my-app:1.0
          ports:
            - containerPort: 8080
```

```
$ kubectl apply -f [DEPLOYMENT_FILE]
```

```
$ kubectl set image deployment
[DEPLOYMENT_NAME] [IMAGE] [IMAGE]:[TAG]
```

```
$ kubectl edit \
  deployment/[DEPLOYMENT_NAME]
```


You can update a Deployment in different ways

[REFRESH](#) [EDIT](#) [DELETE](#) [ACTIONS](#) [KUBECTL](#)

Rolling update

Update workload Pods to a new application version.

Minimum seconds ready

0

?

Maximum surge

25%

?

Maximum unavailable

25%

?

Container images

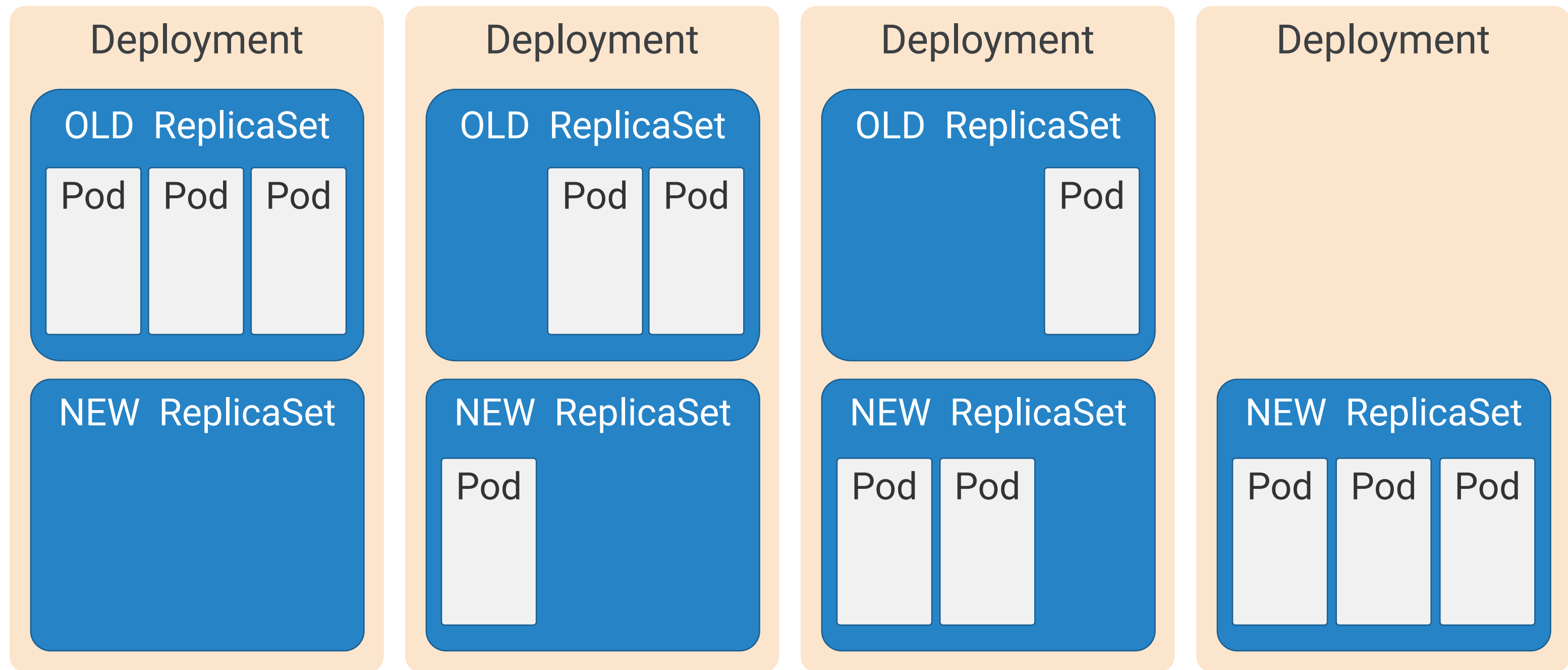
Image of nginx *

nginx:1.7.9

* Indicates required field

[CANCEL](#) [UPDATE](#)

The process behind updating a Deployment

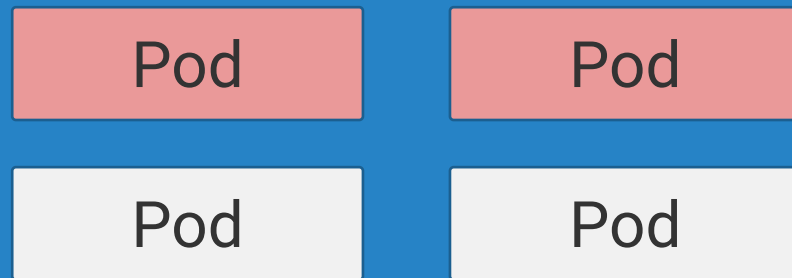


Set parameters for your rolling update strategy

Deployment

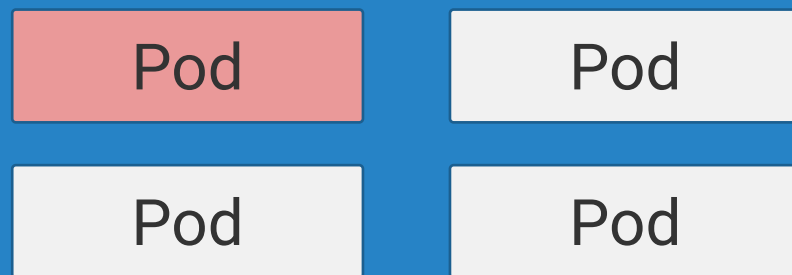
Max unavailable = 2

Old ReplicaSet

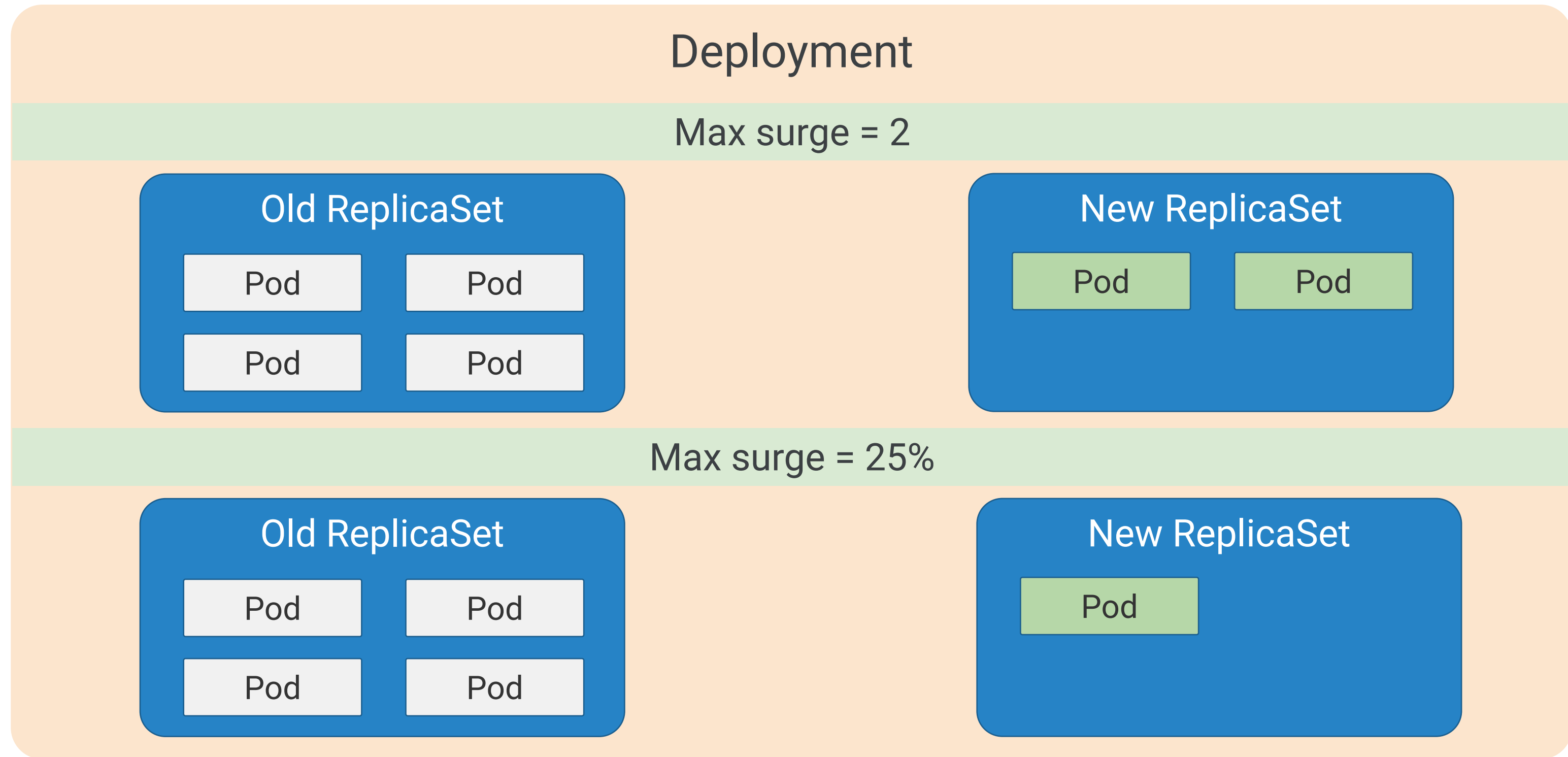


Max unavailable = 25%

Old ReplicaSet



Set parameters for your rolling update strategy



An example of a rolling update strategy

```
[...]
kind: deployment
spec:
  replicas: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 5
      maxUnavailable: 10%
[...]
```


An example of a rolling update strategy

Deployment

Desired Pods = 10 Pods
Max unavailable = 10% of desired Pods
Max surge = 5 Pods

Total Pods = 10

Old ReplicaSet

Number of Pods = 10

An example of a rolling update strategy

Deployment

Desired Pods = 10 Pods
Max unavailable = 10% of desired Pods
Max surge = 5 Pods

Total Pods = 10

Old ReplicaSet

Number of Pods = 10

New ReplicaSet

Number of Pods = 5

An example of a rolling update strategy

Deployment

Desired Pods = 10 Pods
Max unavailable = 10% of desired Pods
Max surge = 5 Pods

Total Pods = 15 (Max)

Old ReplicaSet

Number of Pods = 10

New ReplicaSet

Number of Pods = 5

An example of a rolling update strategy

Deployment

Desired Pods = 10 Pods
Max unavailable = 10% of desired Pods
Max surge = 5 Pods

Total Pods = 15 (Max)

Old ReplicaSet

Number of Pods = $10 - 6 = 4$

New ReplicaSet

Number of Pods = 5

An example of a rolling update strategy

Deployment

Desired Pods = 10 Pods
Max unavailable = 10% of desired Pods
Max surge = 5 Pods

Total Pods = 9 (Min)

Old ReplicaSet

Number of Pods = $10 - 6 = 4$

New ReplicaSet

Number of Pods = **5**

An example of a rolling update strategy

Deployment

Desired Pods = 10 Pods
Max unavailable = 10% of desired Pods
Max surge = 5 Pods

Total Pods = 9 (Min)

Old ReplicaSet

Number of Pods = $10 - 6 = 4$

New ReplicaSet

Number of Pods = $5 + 5 = 10$

An example of a rolling update strategy

Deployment

Desired Pods = 10 Pods
Max unavailable = 10% of desired Pods
Max surge = 5 Pods

Total Pods = 14

Old ReplicaSet

Number of Pods = $10 - 6 = 4$

New ReplicaSet

Number of Pods = $5 + 5 = 10$

An example of a rolling update strategy

Deployment

Desired Pods = 10 Pods
Max unavailable = 10% of desired Pods
Max surge = 5 Pods

Total Pods = 10

Old ReplicaSet

Number of Pods = $4 - 4 = 0$

New ReplicaSet

Number of Pods = $5 + 5 = 10$

An example of a rolling update strategy

Deployment

Desired Pods = 10 Pods
Max unavailable = 10% of desired Pods
Max surge = 5 Pods

Total Pods = 10

New ReplicaSet

Number of Pods = 5 + 5 = 10

Applying a Recreate strategy

```
[...]
kind: deployment
spec:
  replicas: 10
  strategy:
    type: Recreate
[...]
```

Resource requests and limits

Limit:

150 Mb memory
1.0 CPU

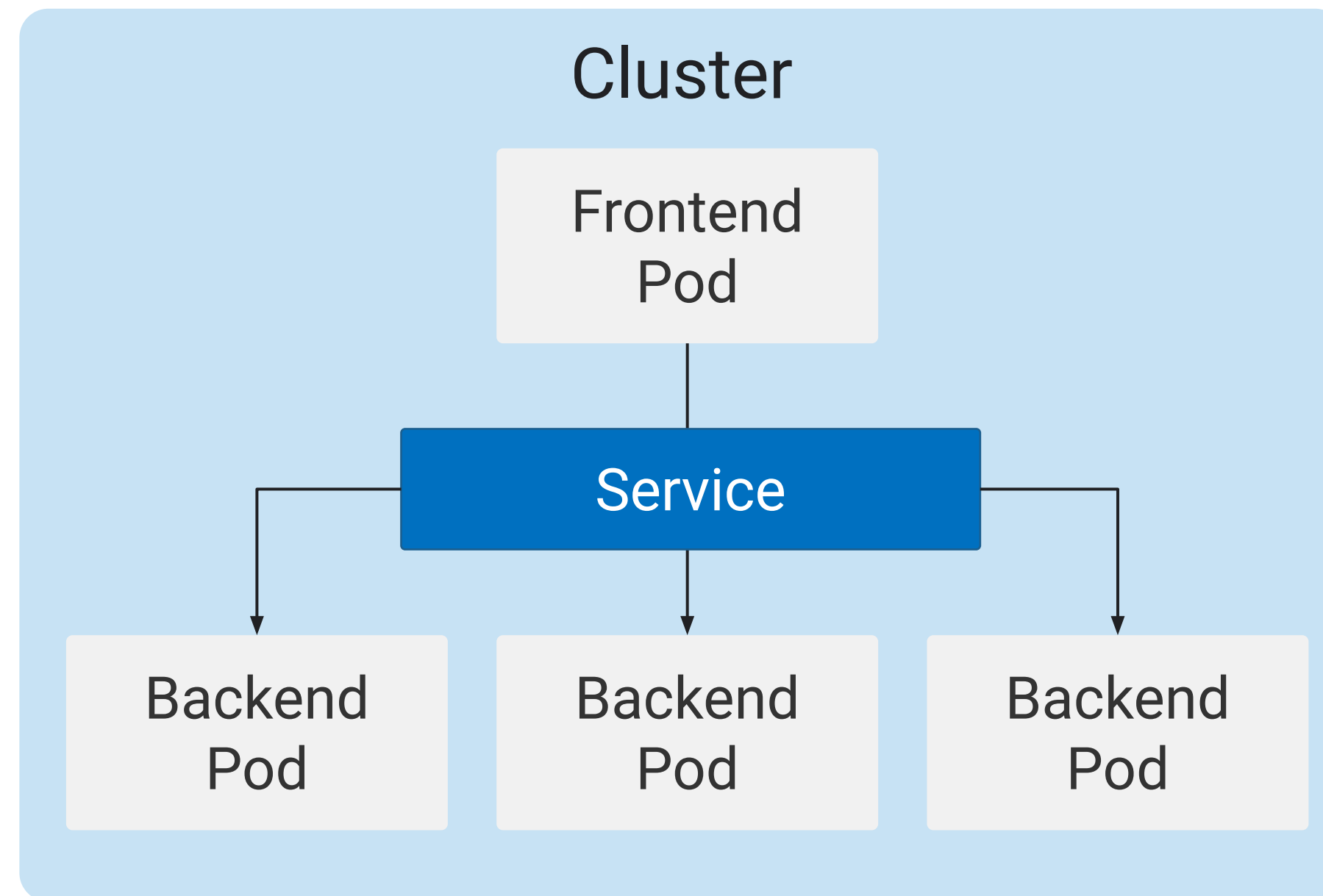
Request

100 Mb memory
0.5 CPU

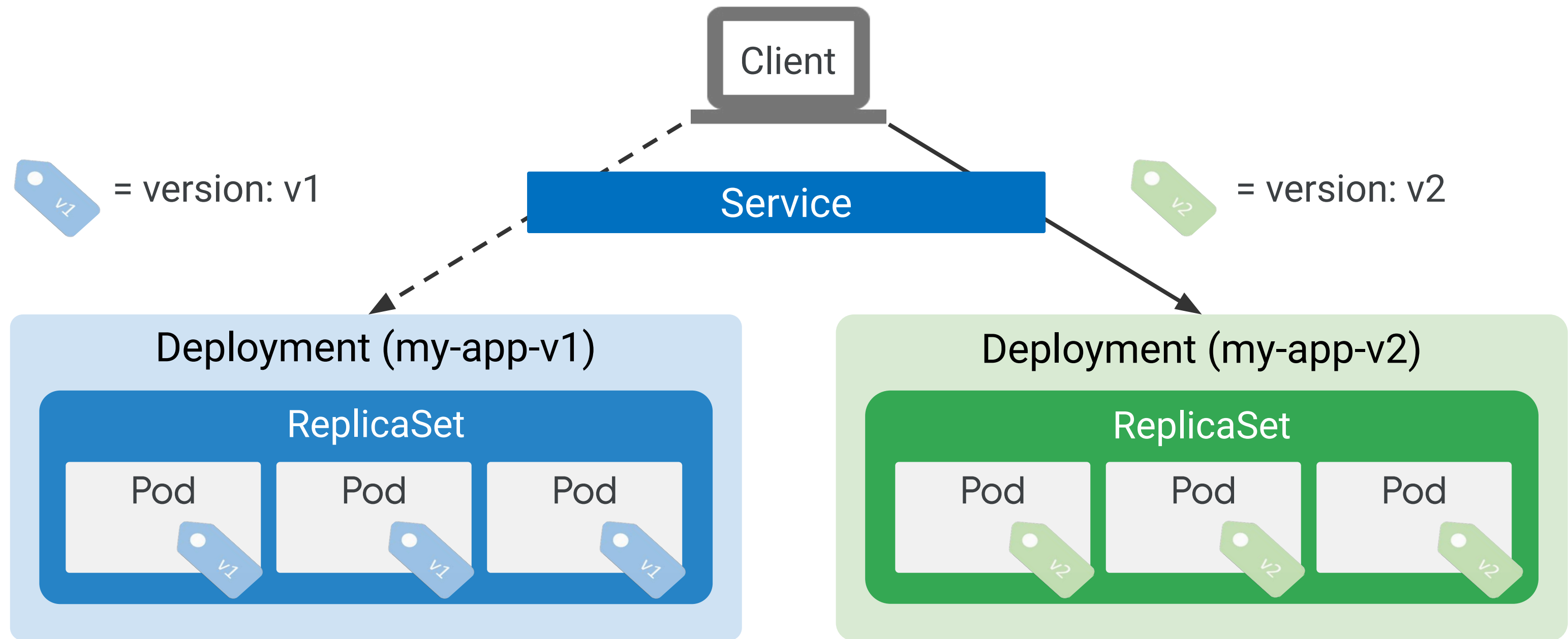
Container settings

```
containers:  
- name: container1  
  image: busybox  
  resources:  
    requests:  
      memory: "32Mi"  
      cpu: "250m"  
    limits:  
      memory: "64Mi"  
      cpu: "500m"
```

Service is a stable network representation of a set of pods



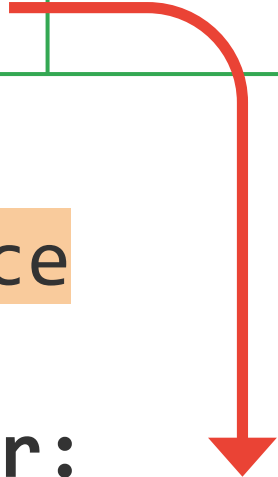
A blue/green deployment strategy ensures app services remain available



Applying a blue/green deployment strategy

```
[...]  
kind: Service  
spec:  
  selector:  
    app: my-app  
    version: v1  
[...]
```

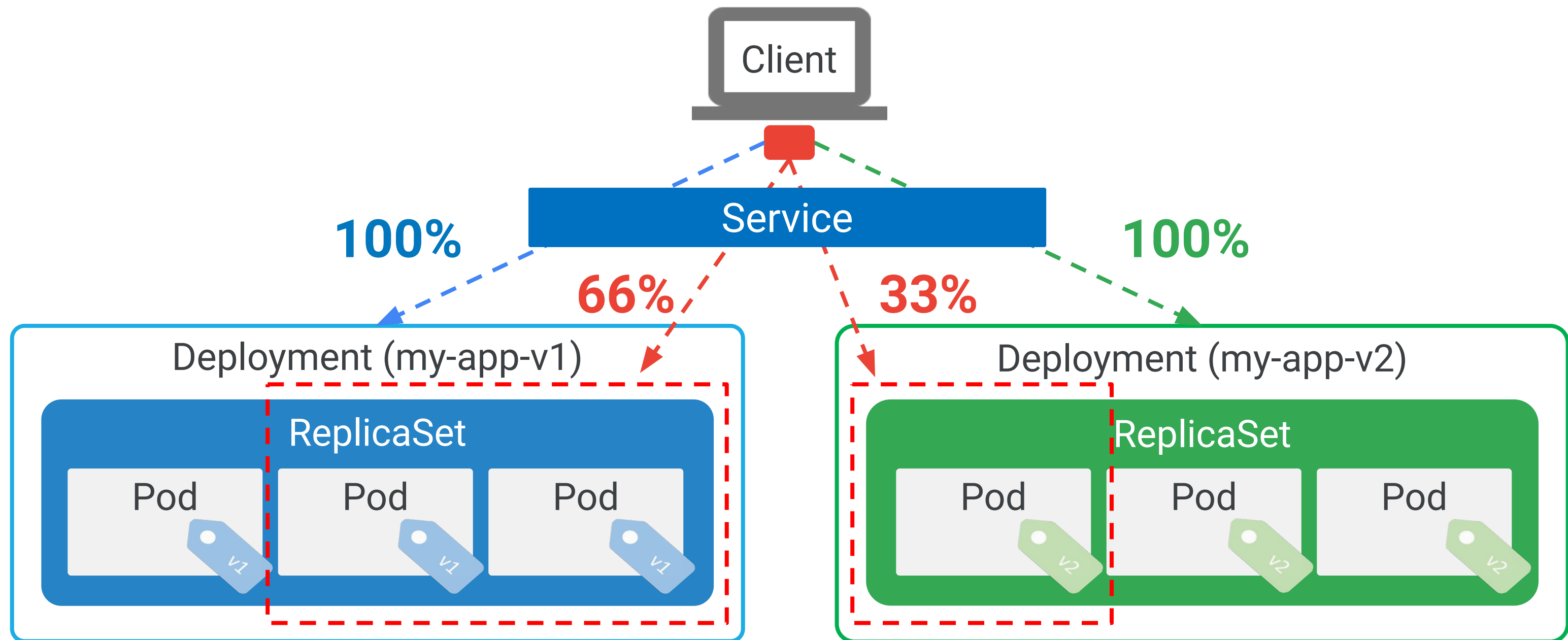
```
[...]  
kind: Service  
spec:  
  selector:  
    app: my-app  
    version: v2  
[...]
```



```
$ kubectl apply -f my-app-v2.yaml
```

```
$ kubectl patch service my-app-service -p \  
'{"spec":{"selector":{"version":"v2"}}}'
```

Canary deployment is an update strategy where traffic is gradually shifted to the new version



Applying a canary deployment

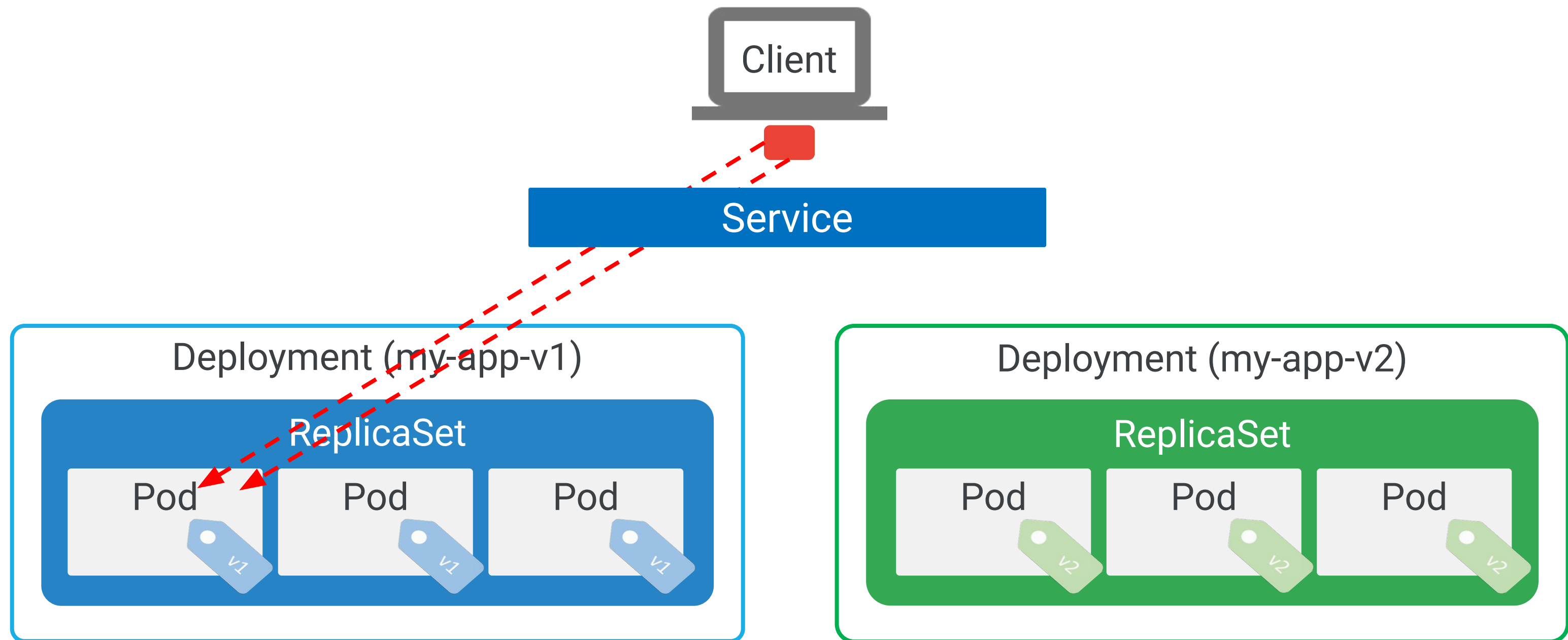
```
[...]
kind: Service
spec:
  selector:
    app: my-app
[...]
```

```
$ kubectl apply -f my-app-v2.yaml
```

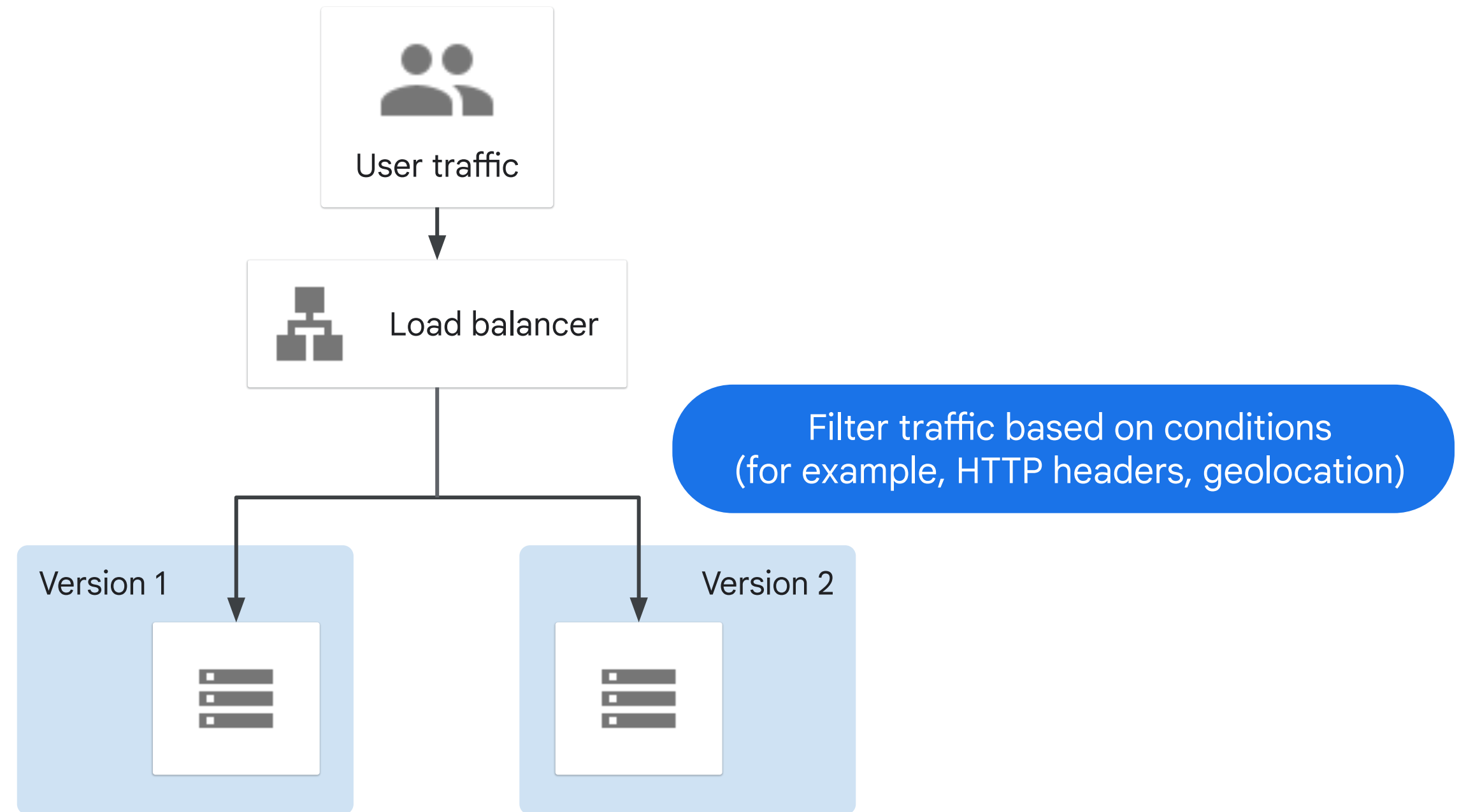
```
$ kubectl scale deploy/my-app-v2 --replicas=10
```

```
$ kubectl delete -f my-app-v1.yaml
```

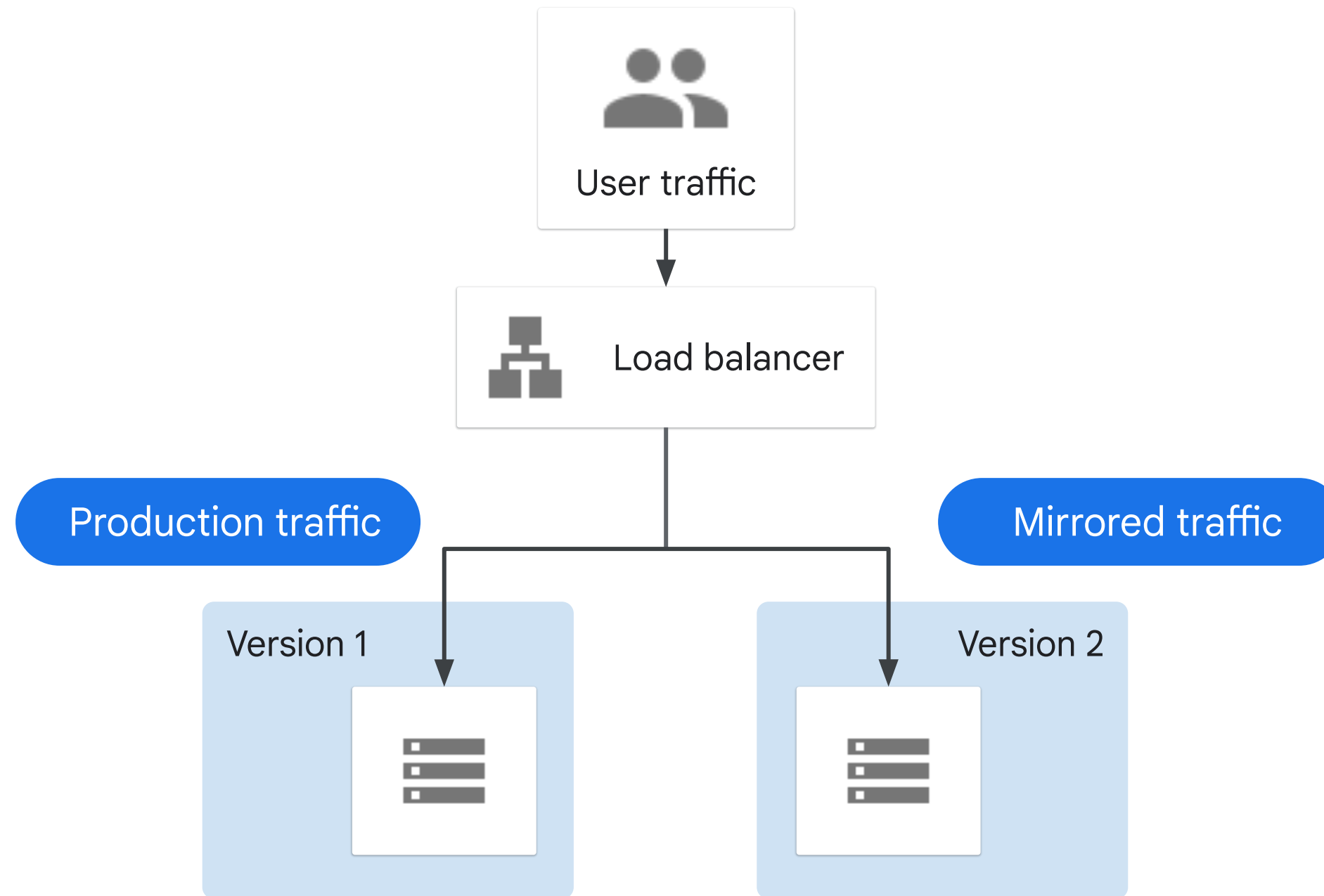
Session affinity ensures that all client requests are sent to the same Pod



A/B testing is used to measure the effectiveness of functionality in an application



Shadow testing allows you to run a new, hidden version



Choosing the right strategy

Deployment or testing pattern	Zero downtime	Real production traffic testing	Releasing to users based on conditions	Rollback duration	Impact on hardware and cloud costs
Recreate Version 1 is terminated, and Version 2 is rolled out.	✗	✗	✗	Fast but disruptive due to downtime	No extra setup required
Rolling update Version 2 is gradually rolled out and replaces Version 1.	✓	✗	✗	Slow	Can require extra setup for surge upgrades
Blue/green Version 2 is released alongside version 1; the traffic is switched to Version 2 after it is tested.	✓	✗	✗	Instant	Need to maintain blue and green environments simultaneously
Canary Version 2 is released to a subset of users, followed by a full rollout.	✓	✓	✗	Fast	No extra setup required
A/B Version 2 is released, under specific conditions, to a subset of users.	✓	✓	✓	Fast	No extra setup required
Shadow Version 2 receives real-world traffic without impacting user requests.	✓	✓	✗	Does not apply	Need to maintain parallel environments in order to capture and replay user requests

Rolling back a Deployment

```
$ kubectl rollout undo deployment [DEPLOYMENT_NAME]
```

```
$ kubectl rollout undo deployment [DEPLOYMENT_NAME] --to-revision=2
```

```
$ kubectl rollout history deployment [DEPLOYMENT_NAME] --revision=2
```

Clean up Policy:

- Default: 10 Revision
- Change: `.spec.revisionHistoryLimit`

Different actions can be applied to a Deployment

Pause

```
$ kubectl rollout pause deployment [DEPLOYMENT_NAME]
```

Resume

```
$ kubectl rollout resume deployment [DEPLOYMENT_NAME]
```

Monitor

```
$ kubectl rollout status deployment [DEPLOYMENT_NAME]
```

Delete a Deployment

```
$ kubectl delete deployment [DEPLOYMENT_NAME]
```

Workloads

REFRESH

DEPLOY

DELETE

Cluster

Namespace

RESET

SAVE

Workloads are deployable units of computing that can be created and managed in a cluster.

OVERVIEW

COST OPTIMIZATION

PREVIEW

Filter

Is system object : False

Filter workloads

☒	Name ↑	Status	Type	Pods	Namespace	Cluster
☒	nginx-deployment	OK	Deployment	3/3	default	standard-cluster-1

Delete resources

Are you sure you want to delete the selected resource?

☒ Delete Horizontal Pod Autoscaler associated with selected Deployment

CANCELDELETE

Agenda

Deployments

Lab: Creating Google Kubernetes Engine Deployments

Jobs and CronJobs

Lab: Deploying Jobs on Google Kubernetes Engine

Cluster Scaling

Controlling Pod Placement

Getting Software into Your Cluster

Lab: Configuring Pod Autoscaling and Node Pools

Summary

Lab Intro

Creating Google Kubernetes
Engine Deployments



Agenda

Deployments

Lab: Creating Google Kubernetes Engine Deployments

Jobs and CronJobs

Lab: Deploying Jobs on Google Kubernetes Engine

Cluster Scaling

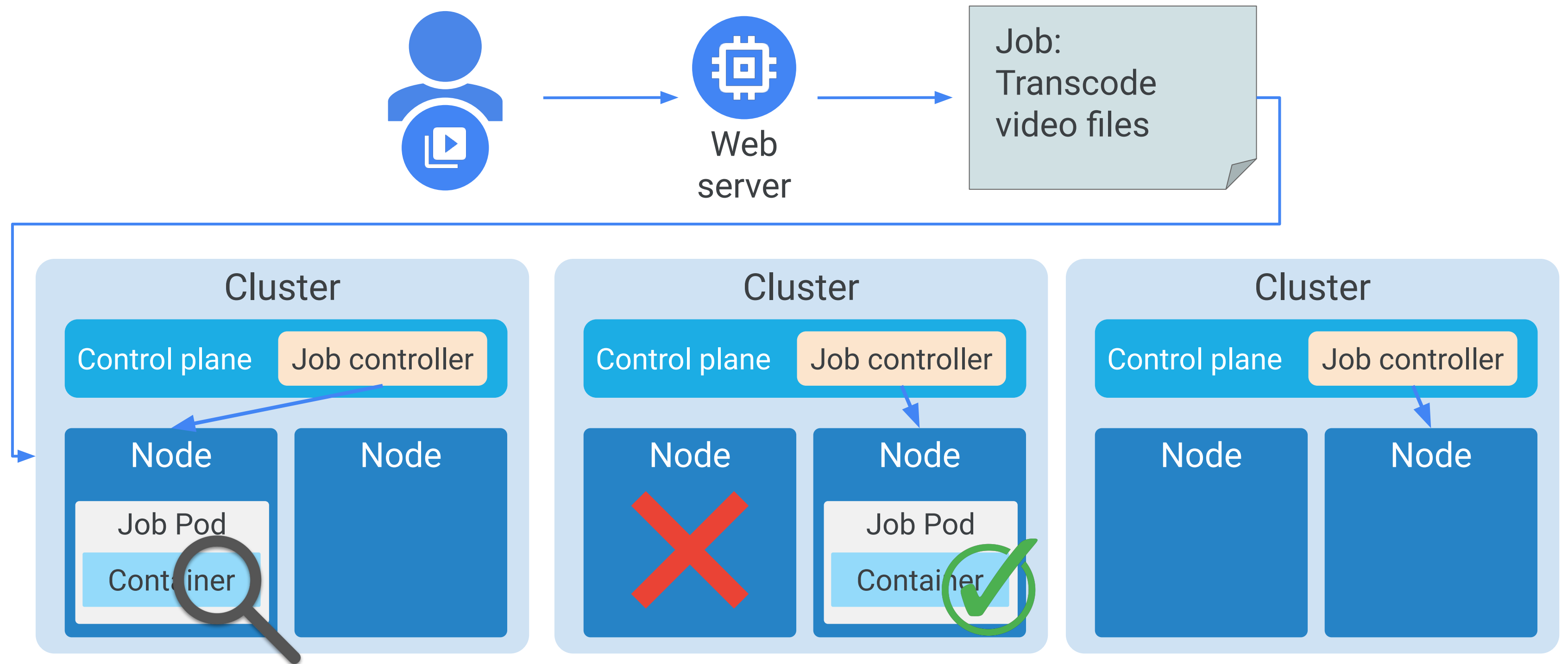
Controlling Pod Placement

Getting Software into Your Cluster

Lab: Configuring Pod Autoscaling and Node Pools

Summary

A scenario where Job provides the solution



Jobs can be defined as non-parallel or parallel

```
apiVersion: batch/v1
kind: Job
metadata:
  name: my-app-job
spec:
  completions: 3
  template:
    spec:
  [...]
```

```
apiVersion: batch/v1
kind: Job
metadata:
  name: my-app-job
spec:
  completions: 3
  parallelism: 2
  template:
    spec:
  [...]
```

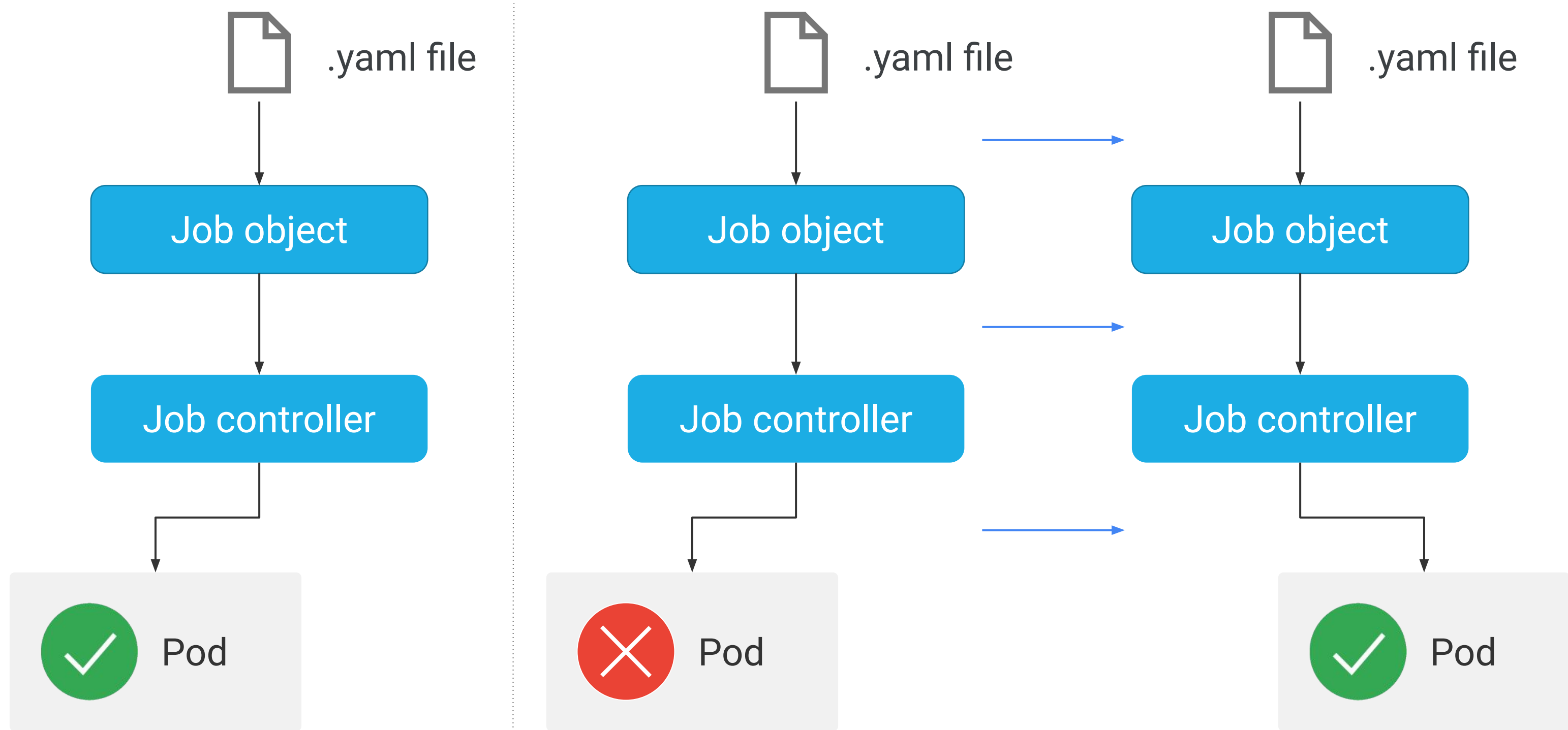
A Job computing π to 2000 places

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  template:
    spec:
      containers:
      - name: pi
        image: perl:5.32
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(2000)"]
        restartPolicy: Never
      backoffLimit: 4
```

```
$ kubectl apply -f [JOB_FILE]
```

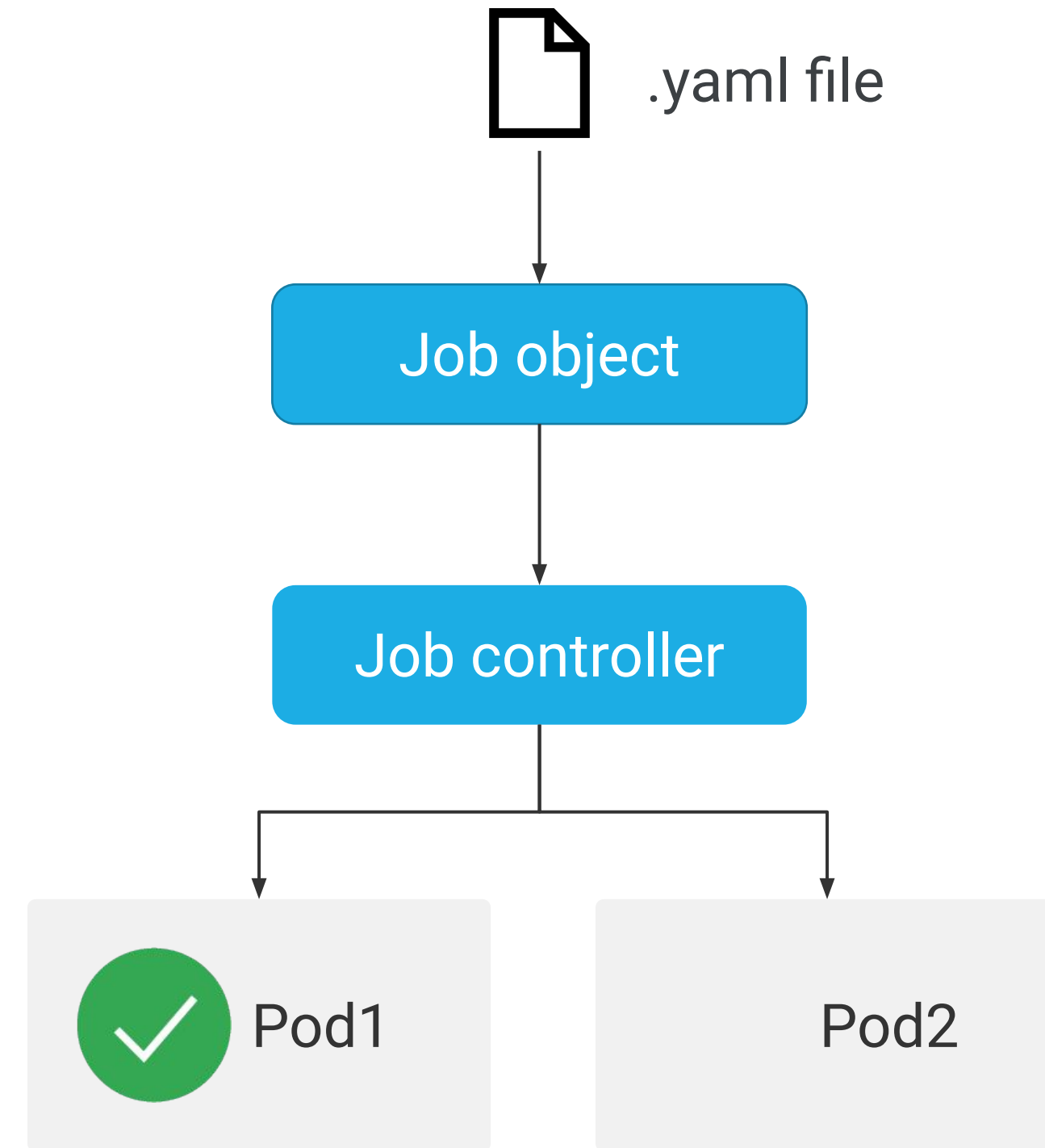
```
$ kubectl run pi --image perl:5.32 --restart Never -- perl -Mbignum=bpi -wle 'print bpi(2000)'
```

The role of a non-parallel Job



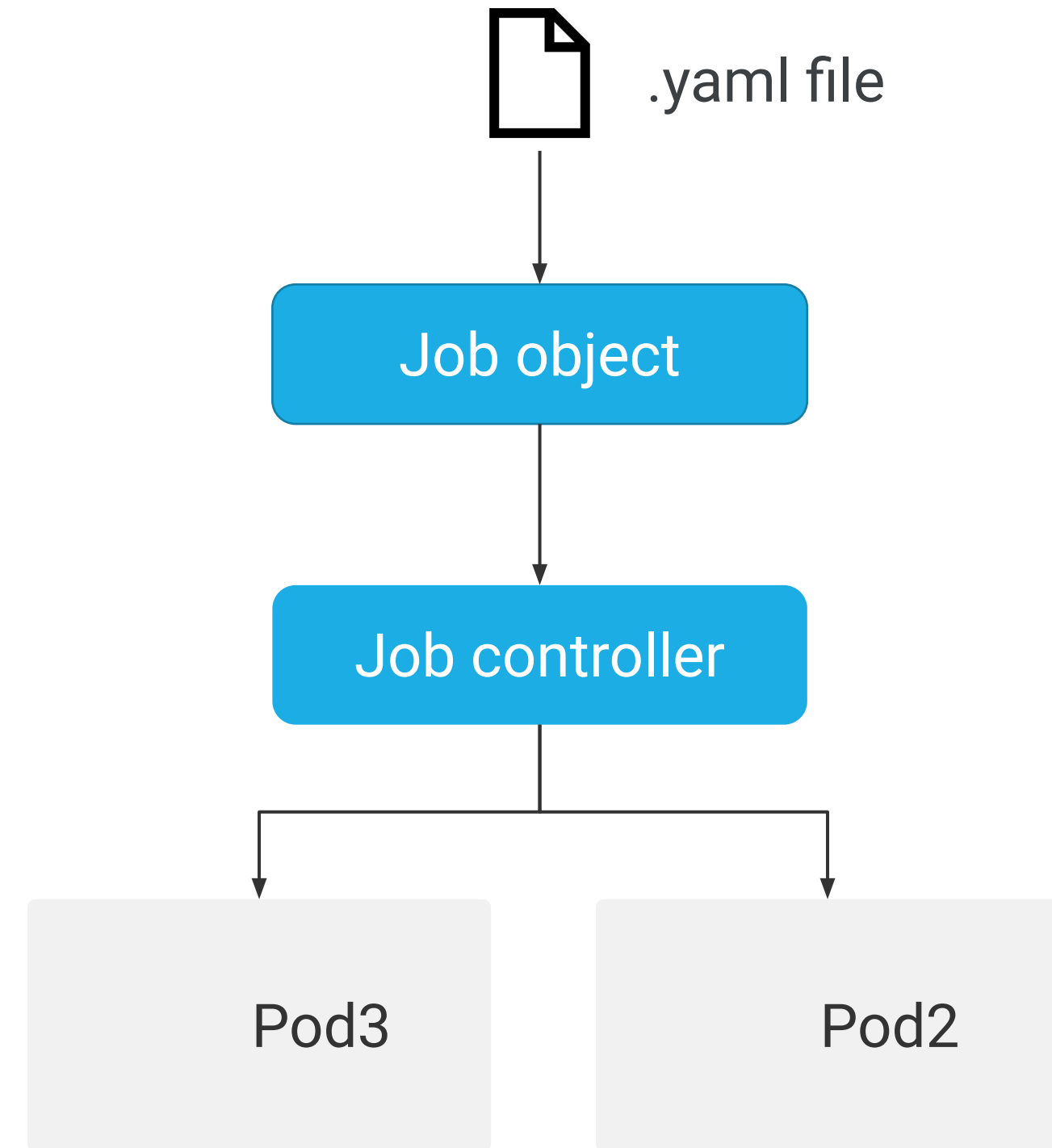
Parallel Job with fixed completion count

```
apiVersion: batch/v1
kind: Job
metadata:
  name: my-app-job
spec:
  completions: 3
  parallelism: 2
  template:
    spec:
  [...]
```



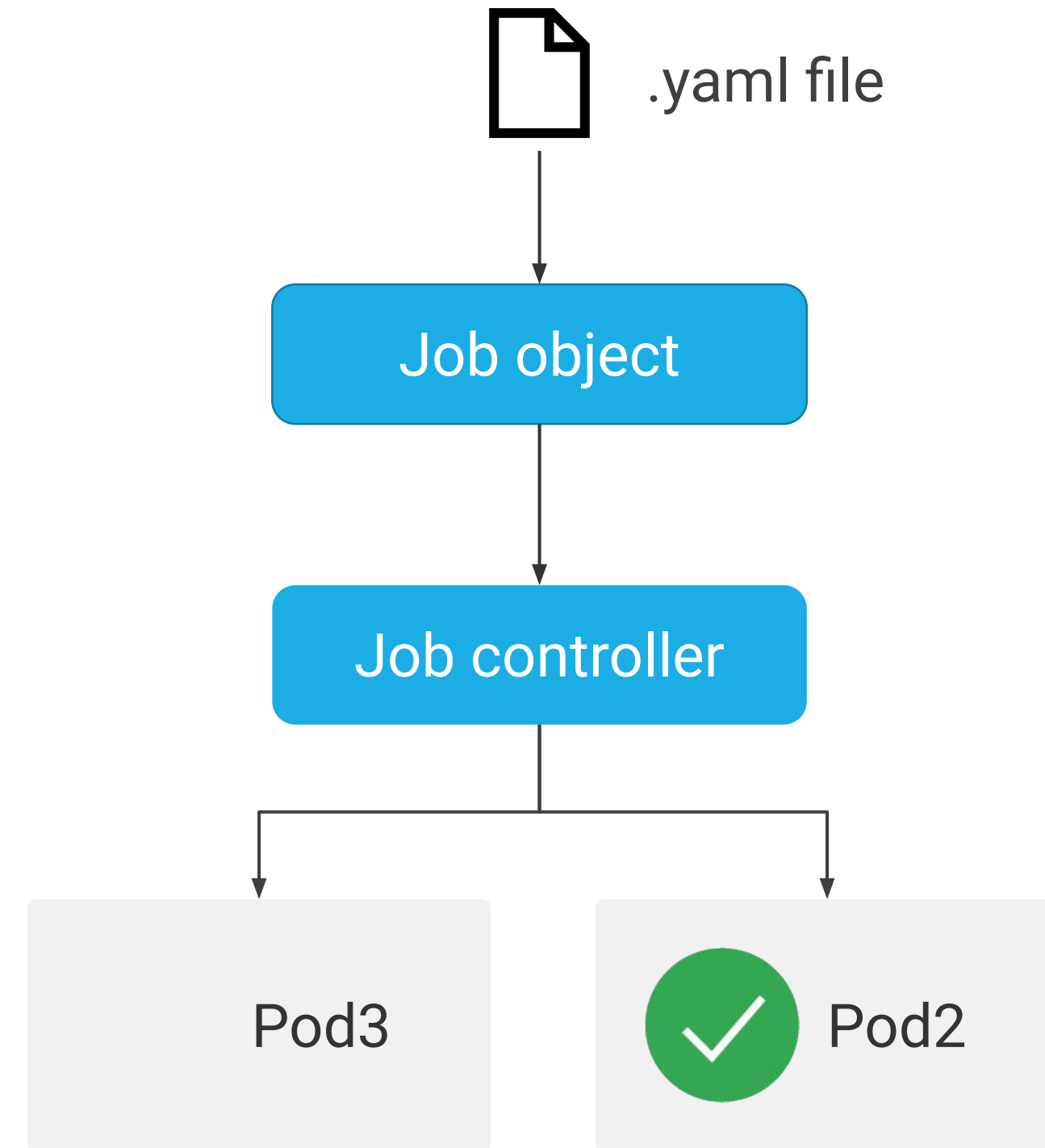
Parallel Job with fixed completion count

```
apiVersion: batch/v1
kind: Job
metadata:
  name: my-app-job
spec:
  completions: 3
  parallelism: 2
  template:
    spec:
  [...]
```



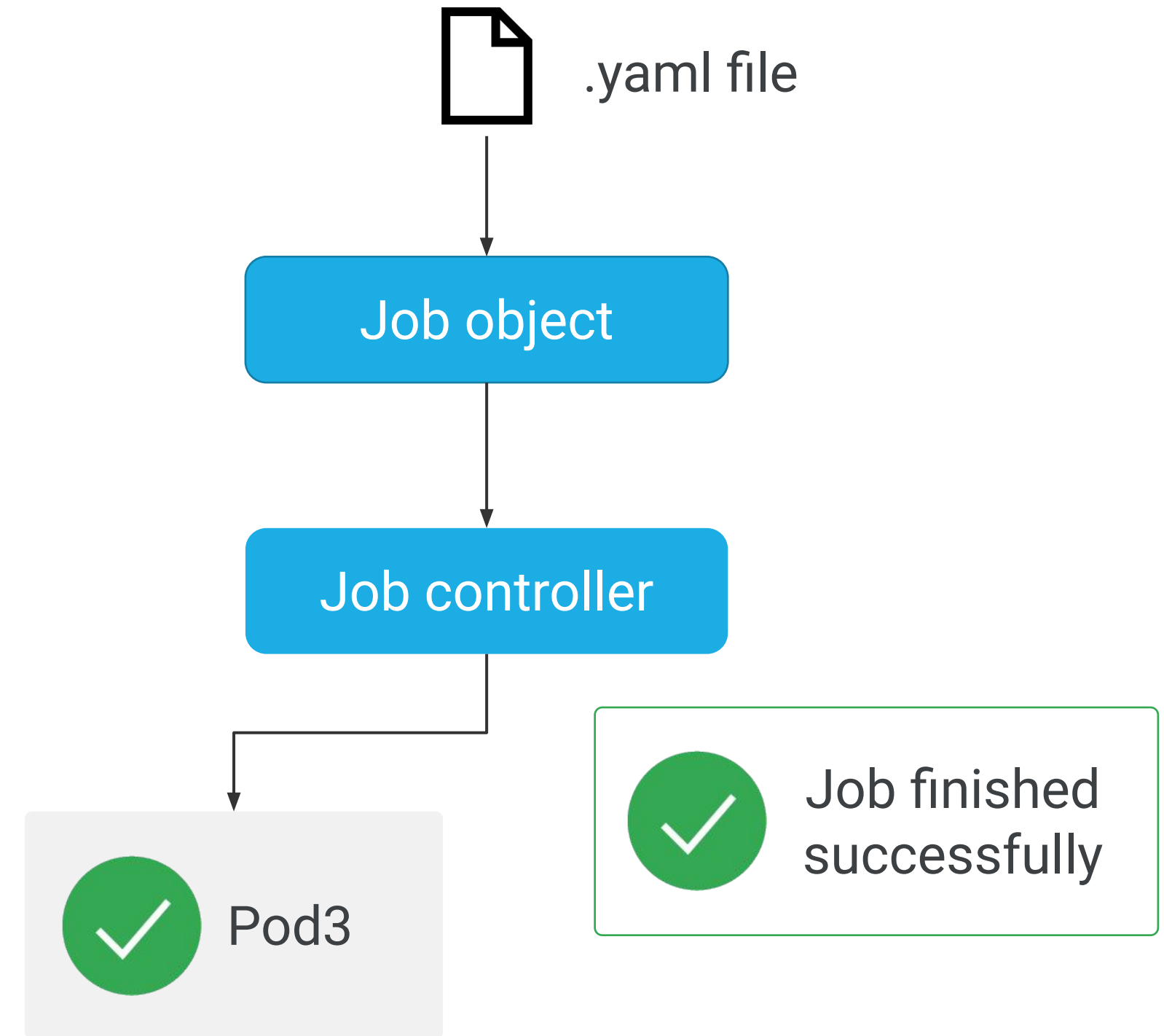
Parallel Job with fixed completion count

```
apiVersion: batch/v1
kind: Job
metadata:
  name: my-app-job
spec:
  completions: 3
  parallelism: 2
  template:
    spec:
  [...]
```



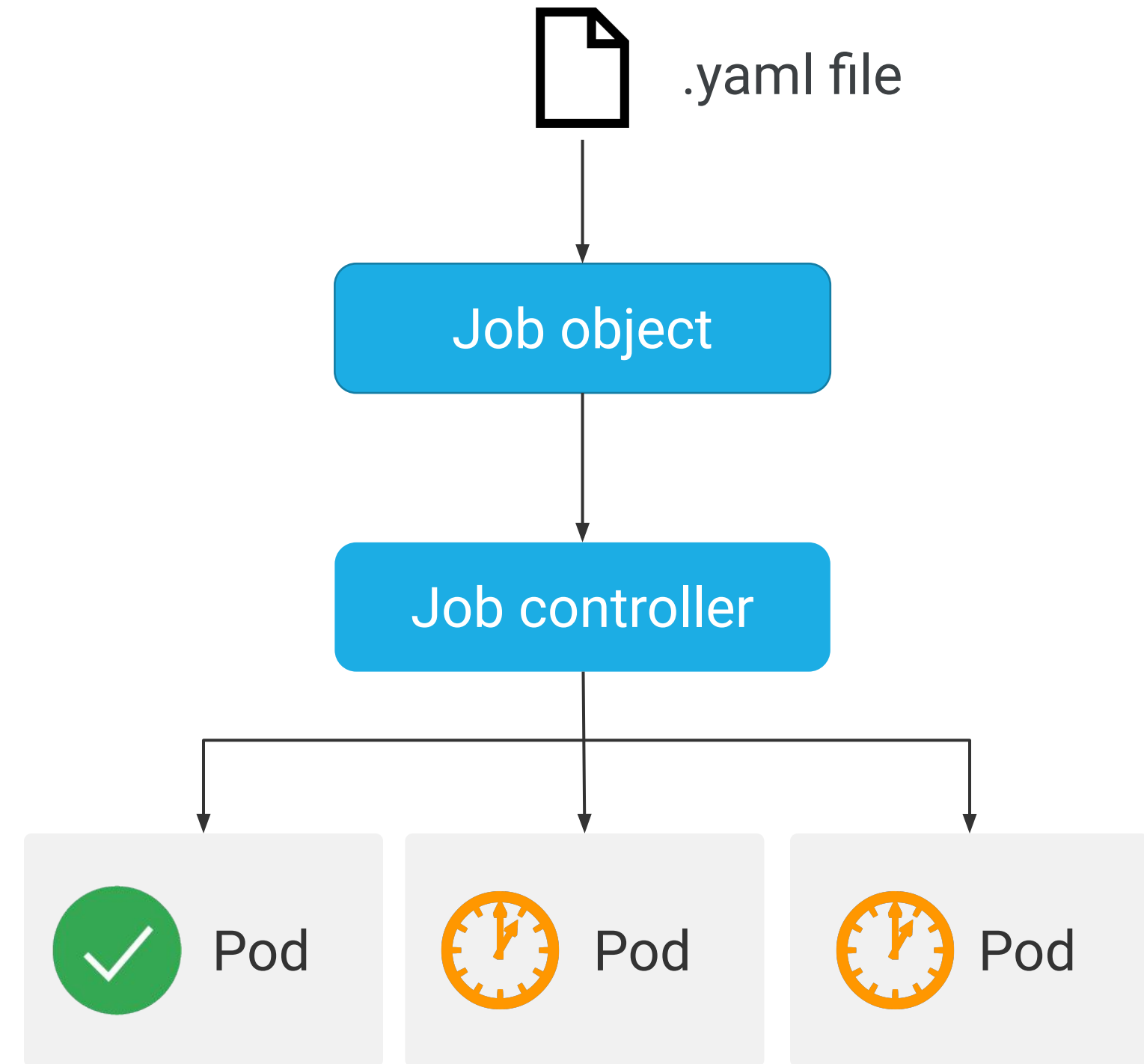
Parallel Job with fixed completion count

```
apiVersion: batch/v1
kind: Job
metadata:
  name: my-app-job
spec:
  completions: 3
  parallelism: 2
  template:
    spec:
  [...]
```



Parallel Job with a worker queue

```
apiVersion: batch/v1
kind: Job
metadata:
  name: my-app-job
spec:
  parallelism: 3
  template:
    spec:
  [...]
```



Inspecting a Job

```
$ kubectl describe job [JOB_NAME]
```

```
$ kubectl get pod -l [job-name=my-app-job]
```

Scaling a Job

```
$ kubectl scale job [JOB_NAME]  
--replicas [VALUE]
```

Scale

Scale a workload to a new size.

Replicas *

* Indicates required field

[CANCEL](#) [SCALE](#)

Failing a Job

```
apiVersion: batch/v1
kind: Job
metadata:
  name: my-app-job
spec:
  backoffLimit: 4
  activeDeadlineSeconds: 300
  template:
  [...]
```

Deleting a Job

```
$ kubectl delete -f [JOB_FILE]
```

```
$ kubectl delete job [JOB_NAME]
```

```
$ kubectl delete job [JOB_NAME]  
--cascade false
```


Cron format is a simple, yet powerful and flexible way to define a schedule

Min	Hour	Day	Mon	Weekday	
*	*	*	*	*	command to be executed
					

Examples	
0 * * * *	Every hour
*/15 * * * *	Every 15 minutes
0 */6 * * *	Every 6 hours
0 20 * * 1-5	Every week Mon-Fri at 8pm
0 0 1 */2 *	At 00:00 on day 1 of every other month

Setting up a cron schedule under the CronJob spec

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: my-app-job
spec:
  schedule: "*/1 * * * *"
  jobtemplate:
    spec:
      template:
        spec:
          [...]

```

Setting up a cron schedule under the CronJob spec

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: my-app-job
spec:
  schedule: "*/1 * * * *"
  startingDeadlineSeconds: 100
  jobtemplate:
    spec:
      template:
        spec:
```

[...]

Setting up a cron schedule under the CronJob spec

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: my-app-job
spec:
  schedule: "*/1 * * * *"
  startingDeadlineSeconds: 100
  concurrencyPolicy: Forbid
  jobtemplate:
    spec:
      template:
        spec:
  [...]
```

Setting up a cron schedule under the CronJob spec

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: my-app-job
spec:
  schedule: "*/1 * * * *"
  startingDeadlineSeconds: 100
  concurrencyPolicy: Forbid
  suspend: True
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobtemplate:
    spec:
  [...]1
```

CronJobs can be managed using kubectl

Create a CronJob

```
$ kubectl apply -f [FILE]
```

Inspect a CronJob

```
$ kubectl describe cronjob [NAME]
```

Delete a CronJob

```
$ kubectl delete cronjob [NAME]
```

Agenda

Deployments

Lab: Creating Google Kubernetes Engine Deployments

Jobs and CronJobs

Lab: Deploying Jobs on Google Kubernetes Engine

Cluster Scaling

Controlling Pod Placement

Getting Software into Your Cluster

Lab: Configuring Pod Autoscaling and Node Pools

Summary

Lab Intro

Deploying Jobs on Google
Kubernetes Engine



Agenda

Deployments

Lab: Creating Google Kubernetes Engine Deployments

Jobs and CronJobs

Lab: Deploying Jobs on Google Kubernetes Engine

Cluster Scaling

Controlling Pod Placement

Getting Software into Your Cluster

Lab: Configuring Pod Autoscaling and Node Pools

Summary

Scaling down a cluster using the gcloud command

```
gcloud container clusters resize  
projectdemo --node-pool default-pool \  
--num_nodes 6
```

Scaling a cluster from the Cloud Console

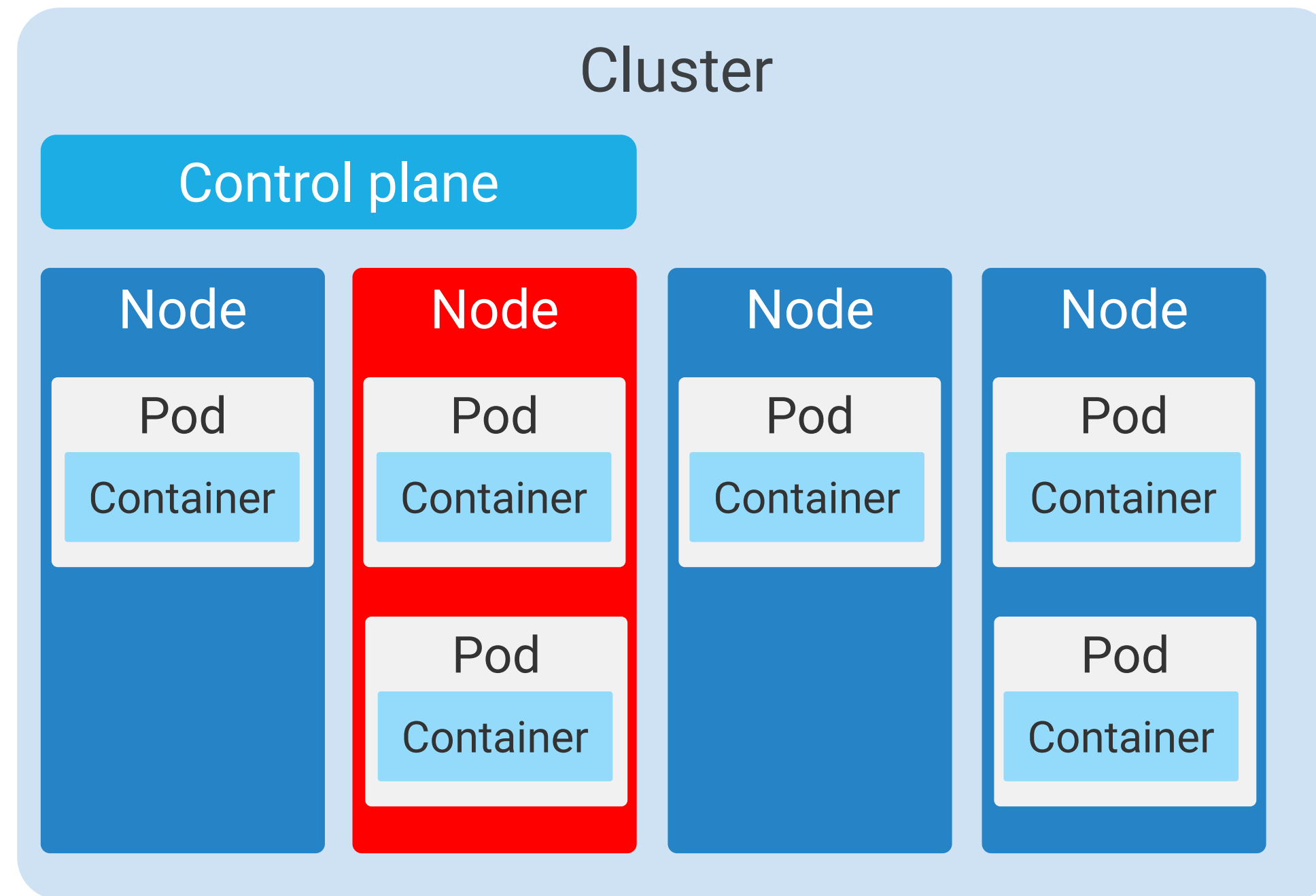
Edit default-pool

Node version
1.20.10-gke.301
[CHANGE](#)

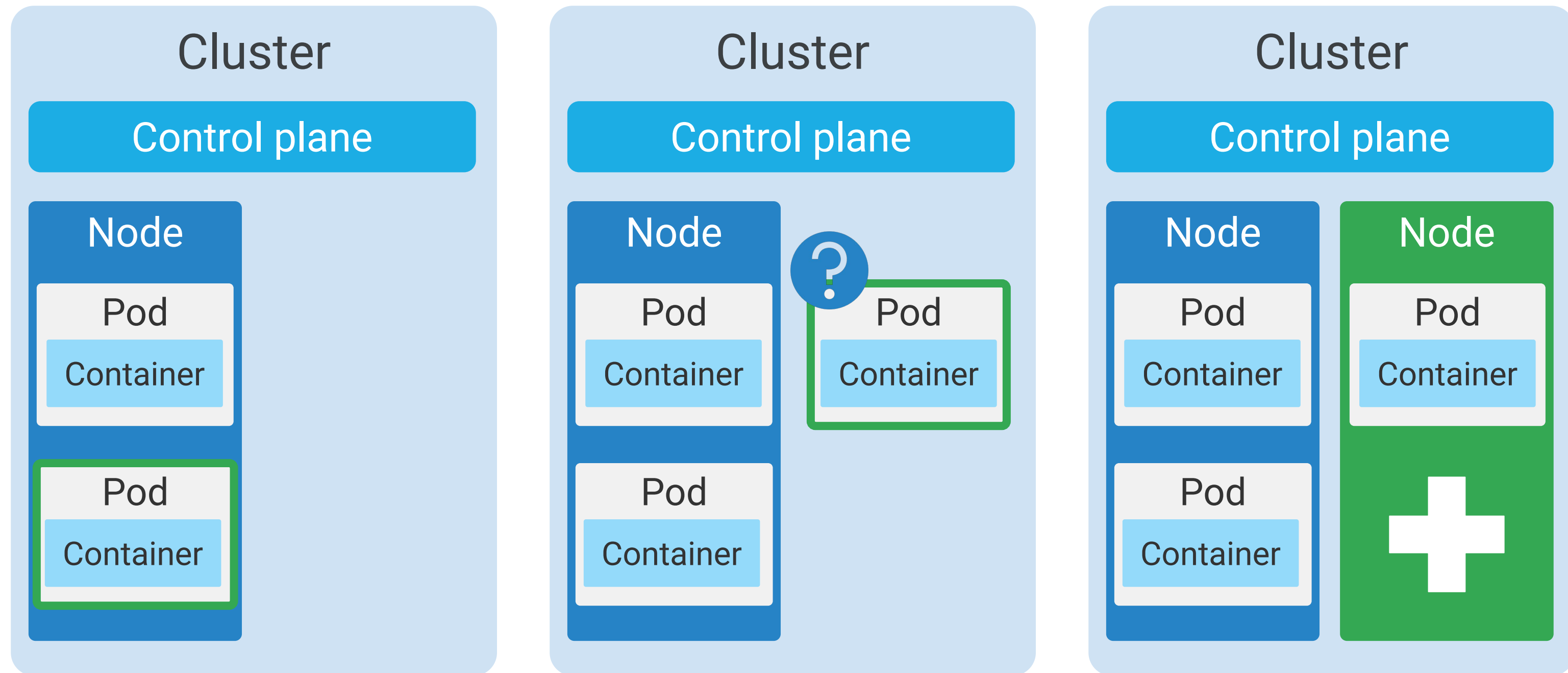
Size
Number of nodes *

☐ **Enable autoscaling** [?](#)

Manual Cluster Scale down selects nodes randomly



Scale up a cluster with autoscaling



Scale down a cluster with autoscaling

- There can be no scale-up events pending.
- Can the node be deleted safely?

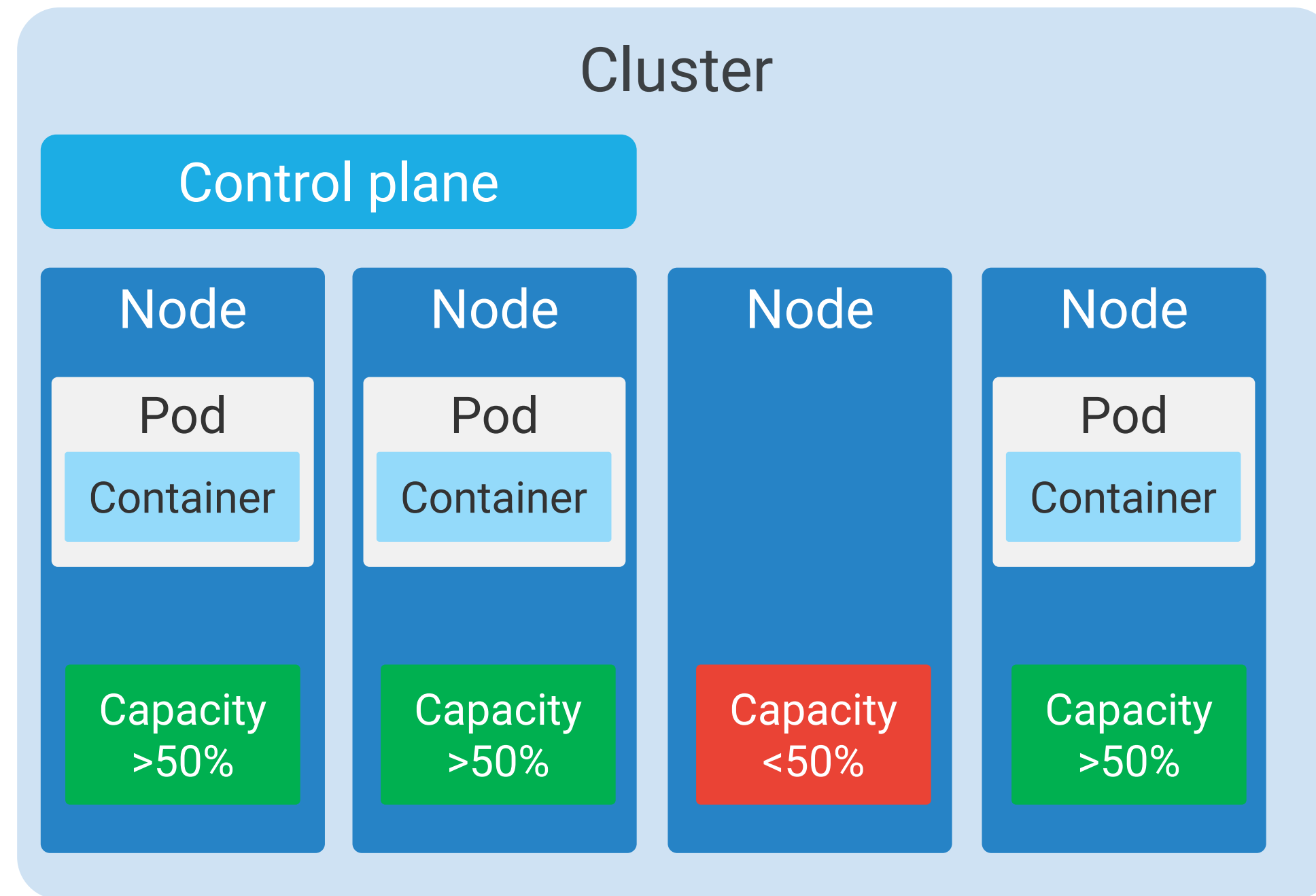
Pod conditions that prevent node deletion

- Not run by a controller.
- Has local storage.
- Restricted by constraint rules.

Pod conditions that prevent node deletion

- `cluster-autoscaler.kubernetes.io/safe-to-evict` is set to False
- Restrictive `PodDisruptionBudget`
- `kubernetes.io/scale-down-disabled` set to True

The Autoscaler removes nodes that remain below 50% utilization



Best practices for working with autoscaled clusters



Don't run Compute Engine autoscaling

Don't manually resize a node using the gcloud command

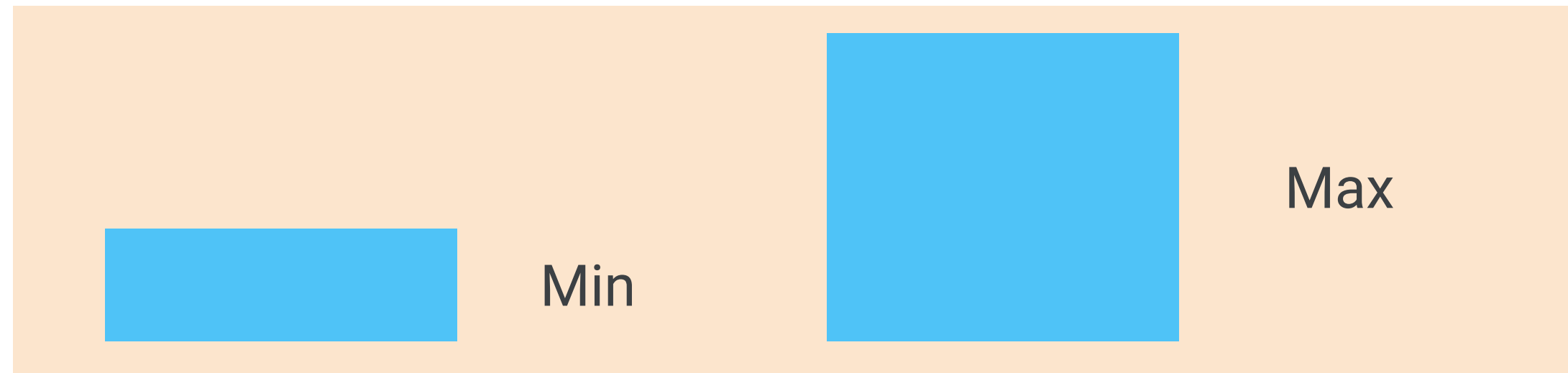
Don't manually modify a node



Specify correct resource requests for Pods

Use PodDisruptionBudget to maintain the app's availability

Setting a node pool size



Node pool = 0
Cluster size $\neq$ 0

MAX = 15,000 nodes x 110 Pods

Increase quota limits to avoid disruption

gcloud commands for autoscaling

Create a cluster with
autoscaling enabled

```
gcloud container clusters create  
[CLUSTER_NAME] --enable-autoscaling  
--min-nodes 15 --max-nodes 50  
[--zone COMPUTE_ZONE]
```

Enable autoscaling for an
existing node pool

```
gcloud container clusters update  
[CLUSTER_NAME] --enable-autoscaling \  
--min-nodes 1 --max-nodes 10 --zone  
[COMPUTE_ZONE] --node-pool [POOL_NAME]
```

Add a node pool with
autoscaling enabled

```
gcloud container node-pools create  
[POOL_NAME] --cluster [CLUSTER_NAME]  
--enable-autoscaling --min-nodes 15  
--max-nodes 50 [--zone COMPUTE_ZONE]
```

Disable autoscaling for an
existing node pool

```
gcloud container clusters update  
[CLUSTER_NAME] --no-enable-autoscaling \  
--node-pool [POOL_NAME] [--zone  
[COMPUTE_ZONE] --project [PROJECT_ID]]
```

Agenda

Deployments

Lab: Creating Google Kubernetes Engine Deployments

Jobs and CronJobs

Lab: Deploying Jobs on Google Kubernetes Engine

Cluster Scaling

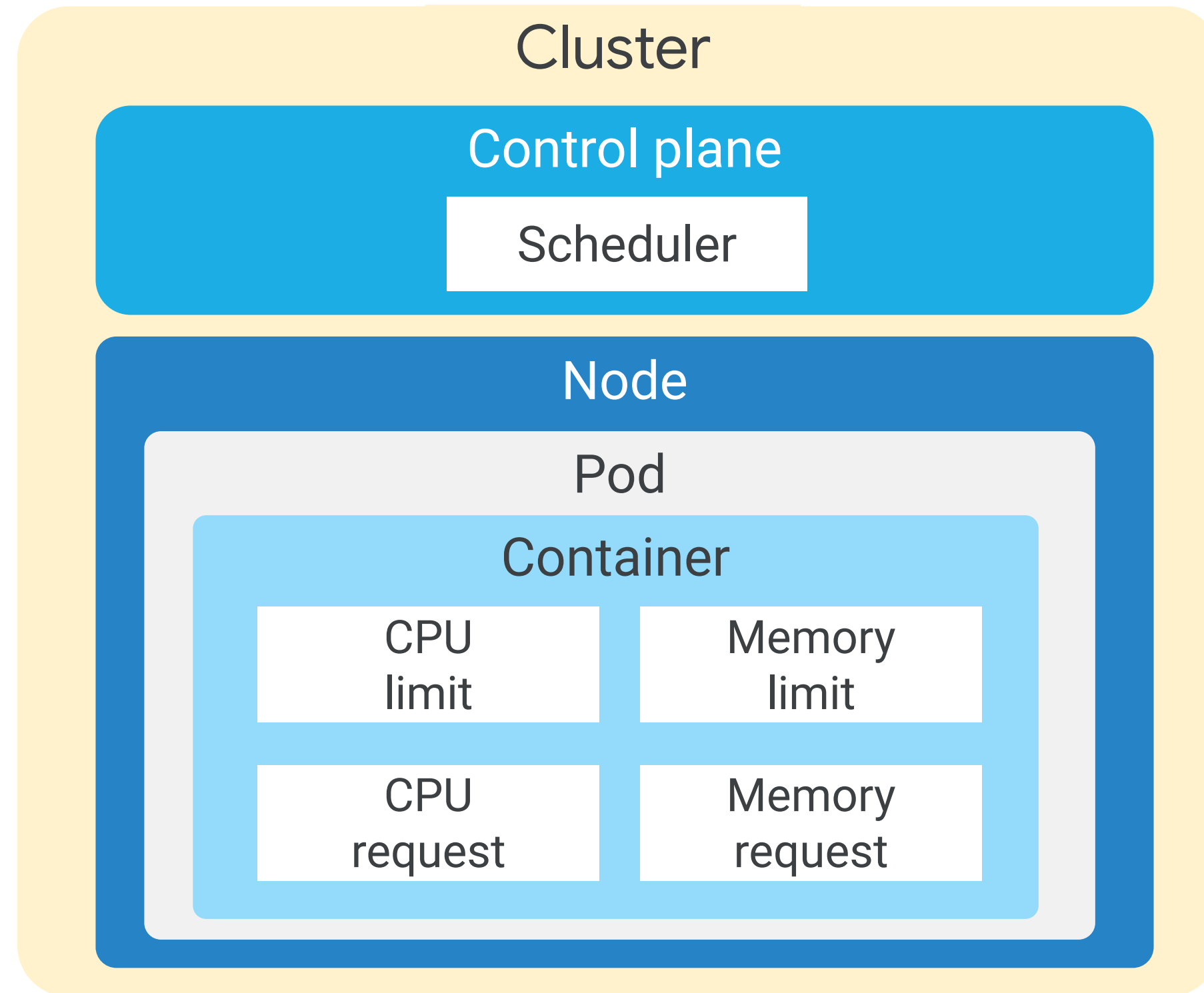
Controlling Pod Placement

Getting Software into Your Cluster

Lab: Configuring Pod Autoscaling and Node Pools

Summary

Controlled scheduling



Nodes must match all the labels present under the nodeSelector field

```
apiVersion: v1
kind: Pod
metadata:
  name: mysql
  labels:
    env: test
spec:
  containers:
  - name: mysql
    image: mysql
    imagePullPolicy: IfNotPresent
  nodeSelector:
    disktype: ssd
[...]
```

```
apiVersion: v1
kind: Node
metadata:
  name: node1
  labels:
    disktype: ssd
[...]
```

Nodes must match all the labels present under the nodeSelector field

```
apiVersion: v1
kind: Pod
metadata:
  name: mysql
  labels:
    env: test
spec:
  containers:
  - name: mysql
    image: mysql
    imagePullPolicy: IfNotPresent
  nodeSelector:
    cloud.google.com/gke-nodepool=ssd
[...]
```


Node affinity is conceptually similar to nodeSelector

```
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: beta.kubernetes.io/instance-type
                operator: In
                values:
                  - n1-highmem-4
                  - n1-highmem-8
[...]
```

Affinity and anti-affinity rules

```
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: accelerator-type
                operator: In
              values:
                - gpu
                - tpu
```

Defining the intensity of preference

```
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 90
          preference:
            - matchExpressions:
                - key: cloud.google.com/gke-nodepool
                  operator: In
                  values:
                    - n1-highmem-4
                    - n1-highmem-8
```


Inter-pod affinity and anti-affinity features are built on the node affinity concept

```
[...]
metadata:
  name: with-pod-affinity
spec:
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchExpressions:
                - key: app
                  operator: In
                  values:
                    - webserver
            topologyKey: failure-domain.beta.kubernetes.io/zone
```

Combining inter-pod affinity and anti-affinity

Pod (#1)
(Label – app:webserver)

Requirement: No app:webserver
on the **same zone**

Preference: Prefer app:cache
on the **same node**

Pod (#2)
(Label – app:webserver)

Requirement: No app:webserver
on the same zone

Preference: Prefer app:cache
on the same node

Pod (#3)
(Label – app:cache)

Requirement: No app:cache
on the **same zone**

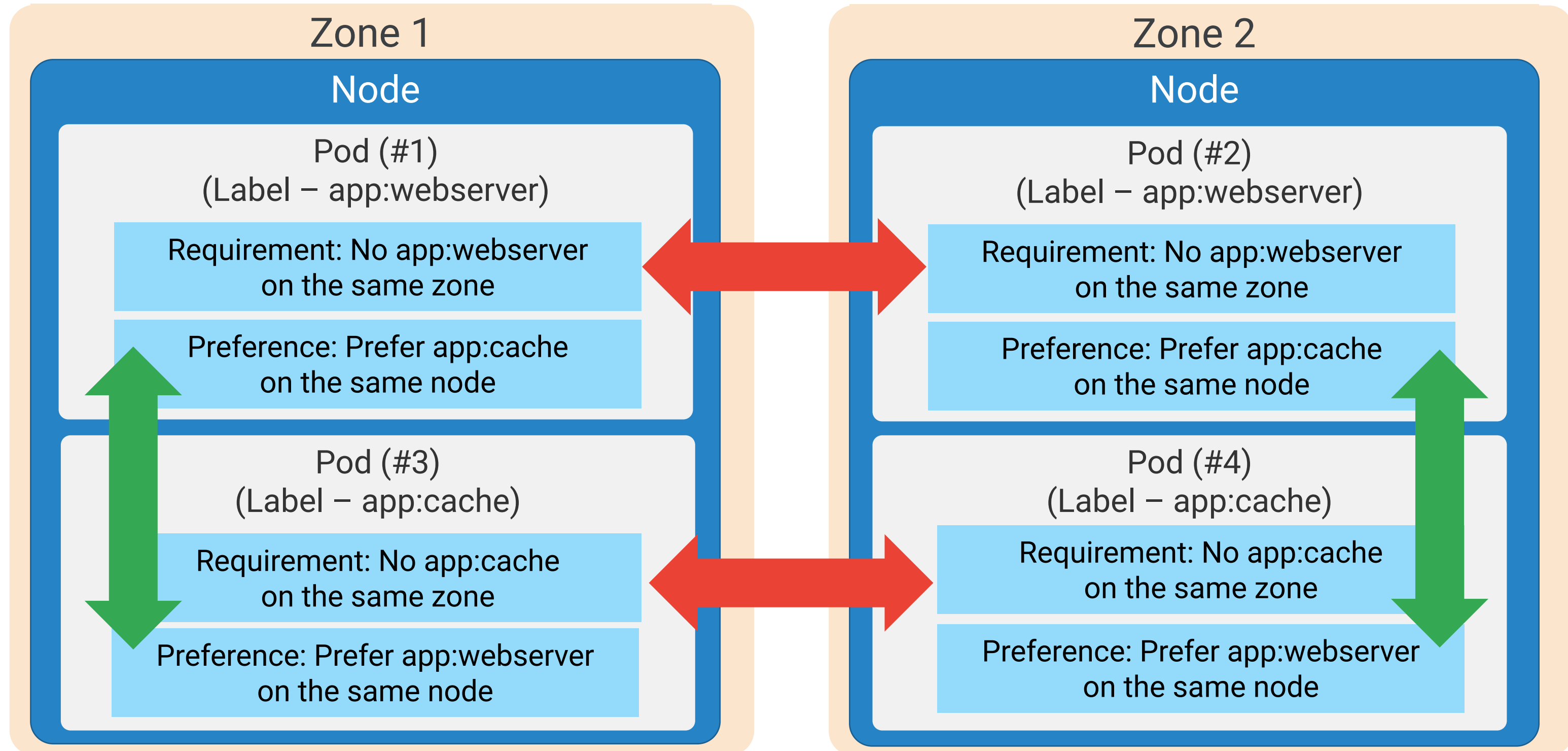
Preference: Prefer app:webserver
on the **same node**

Pod (#4)
(Label – app:cache)

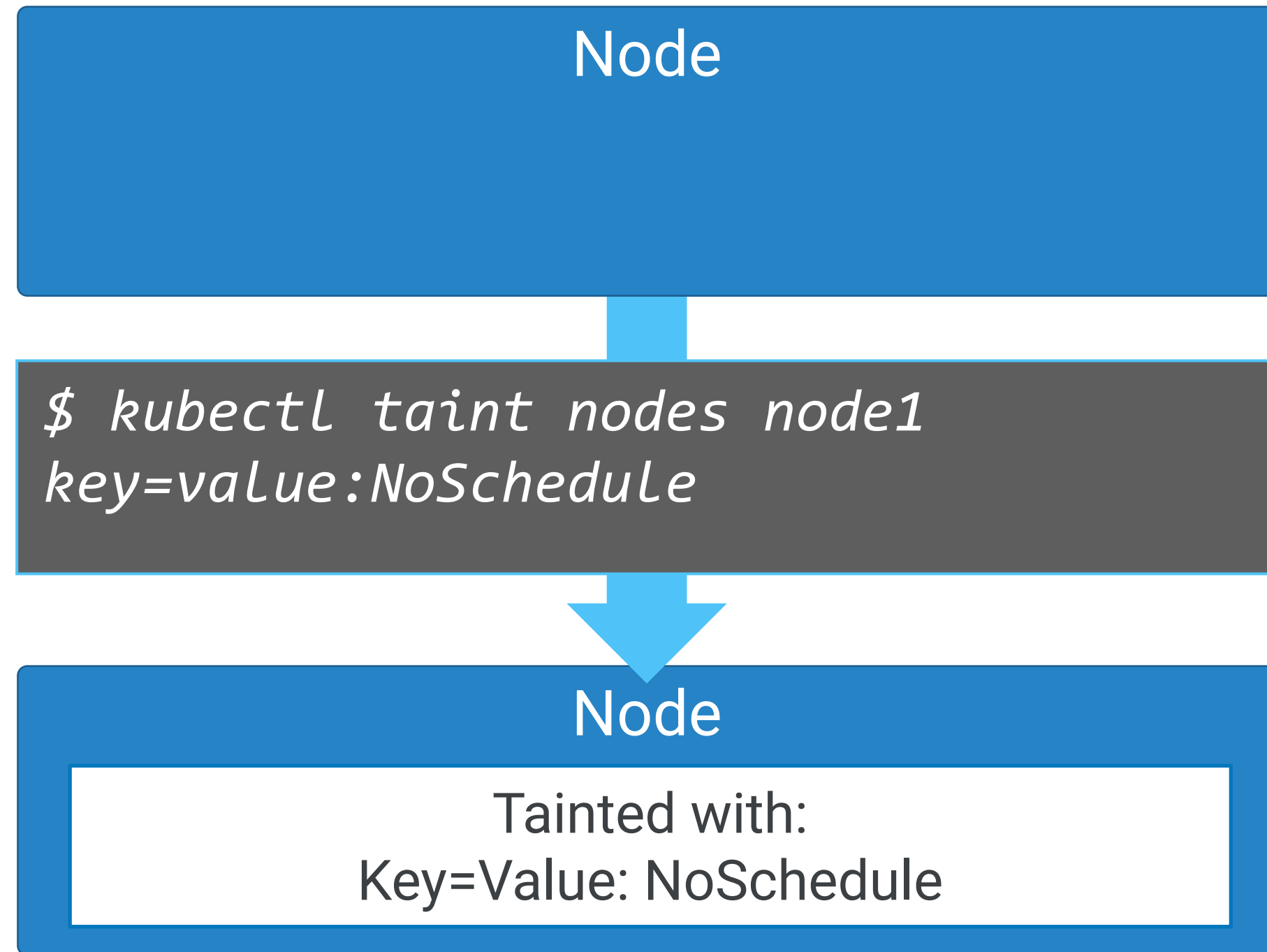
Requirement No app:cache
on the same zone

Preference: Prefer app:webserver
on the same node

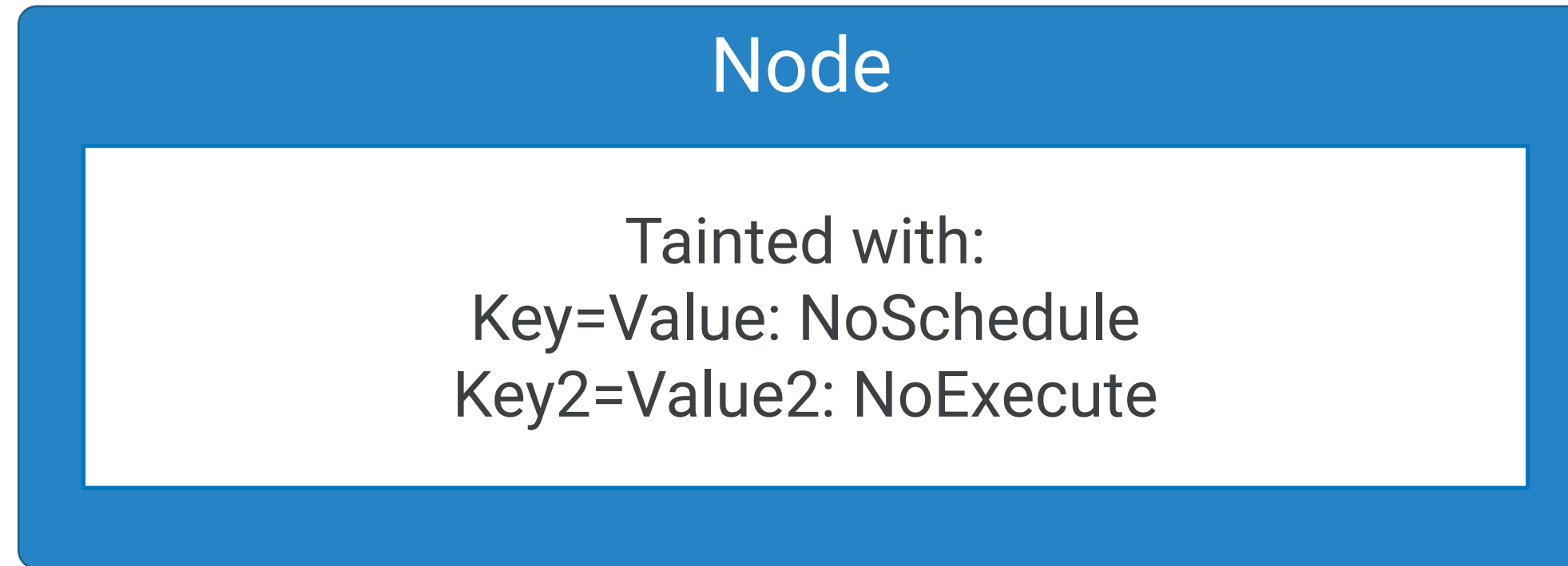
Combining inter-pod affinity and anti-affinity



Taints allow a node to repel Pods



You can apply multiple taints to a node



You can configure Pods to tolerate taints

Node

Tainted with:
Key=Value: NoSchedule

Pod

tolerations:
- key: "key"
operator: "Equal"
value: "value"
effect: "NoSchedule"

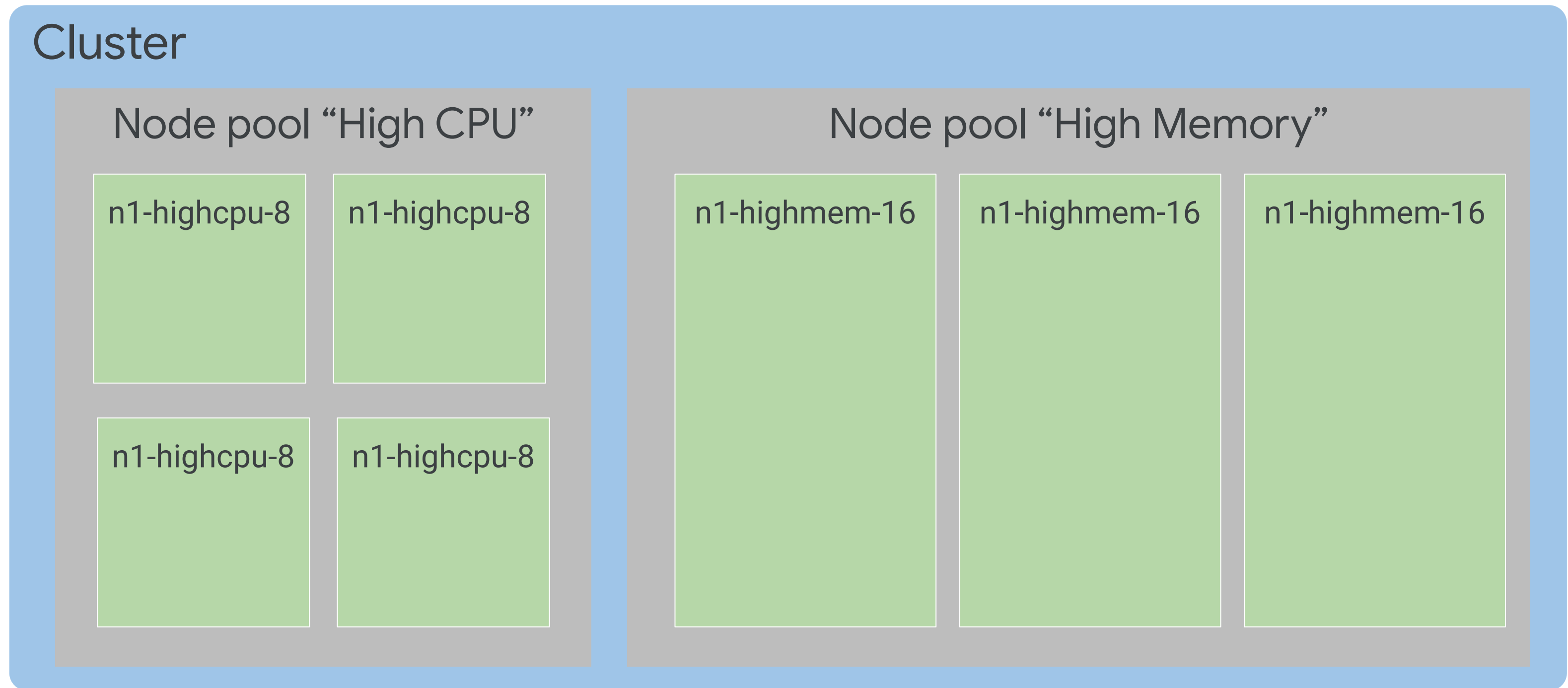
Pod

tolerations:
- key: "key"
operator: "Exists"
effect: "NoSchedule"

Pod

tolerations:
- key: "key"
operator: "Exists"
effect: "PreferNoSchedule"

Use node pools to manage different kinds of nodes



Direct Pods to desired nodes using nodeSelectors

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
  nodeSelector:
    cloud.google.com/gke-nodepool: HighCPU
[...]
```

Or use node pool names with affinity, anti-affinity rules

```
apiVersion: v1
kind: Pod
metadata:
  name: rendering-engine
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: cloud.google.com/gke-nodepool
                operator: NotIn
                values:
                  - HighCPU
```

Agenda

Deployments

Lab: Creating Google Kubernetes Engine Deployments

Jobs and CronJobs

Lab: Deploying Jobs on Google Kubernetes Engine

Cluster Scaling

Controlling Pod Placement

Getting Software into Your Cluster

Lab: Configuring Pod Autoscaling and Node Pools

Summary

How to get software

- Build it yourself, and supply your own YAML.
- Use Helm to install software into your cluster.

Organize Kubernetes objects in packages and deploy complex packages



kubernetes

Charts

Create

Version

Share

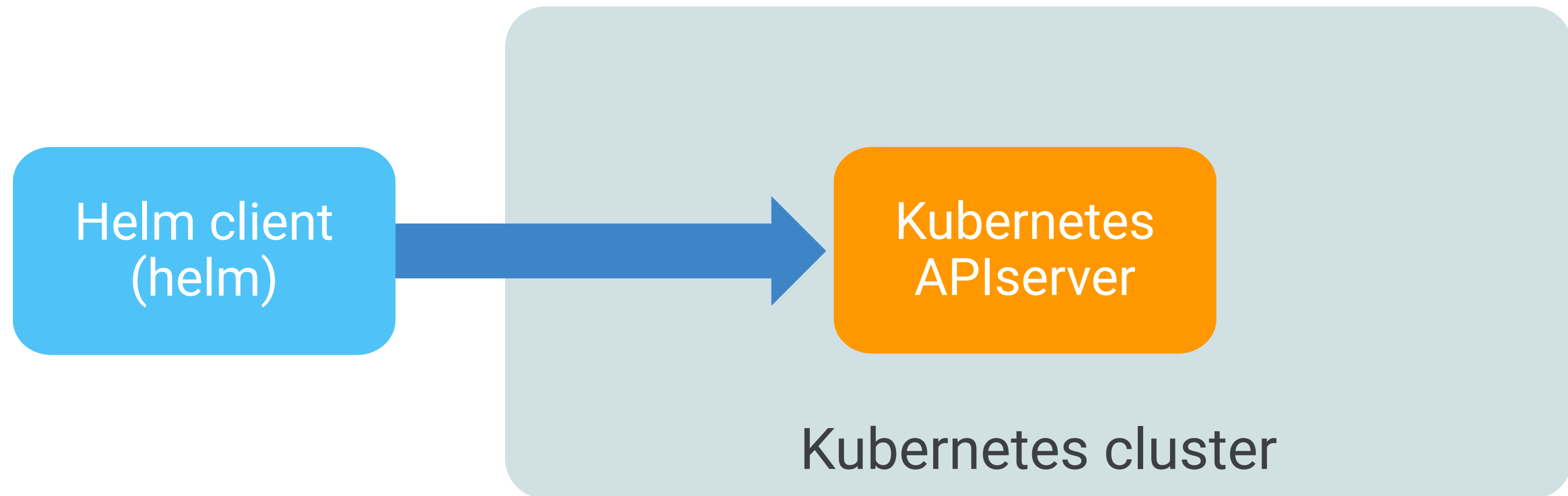
Publish

Download

Configure

Release

Helm interacts directly with the Kubernetes APIserver



How to get software

- Build it yourself, and supply your own YAML.
- Use Helm to install software into your cluster.
- Use Google Cloud Marketplace to install both open-source and commercial software.

Agenda

Deployments

Lab: Creating Google Kubernetes Engine Deployments

Jobs and CronJobs

Lab: Deploying Jobs on Google Kubernetes Engine

Cluster Scaling

Controlling Pod Placement

Getting Software into Your Cluster

Lab: Configuring Pod Autoscaling and Node Pools

Summary

Lab Intro

Configuring Pod Autoscaling and
Node Pools



Agenda

Deployments

Lab: Creating Google Kubernetes Engine Deployments

Jobs and CronJobs

Lab: Deploying Jobs on Google Kubernetes Engine

Cluster Scaling

Controlling Pod Placement

Getting Software into Your Cluster

Lab: Configuring Pod Autoscaling and Node Pools

Summary

Summary

Create and use Deployments.

Create and run Jobs and CronJobs.

Use Helm Charts.

Scale clusters manually and automatically.

Configure Node and Pod affinity.

